

Mattermost Cross Guard

Cross-domain message relay for Mattermost Federal

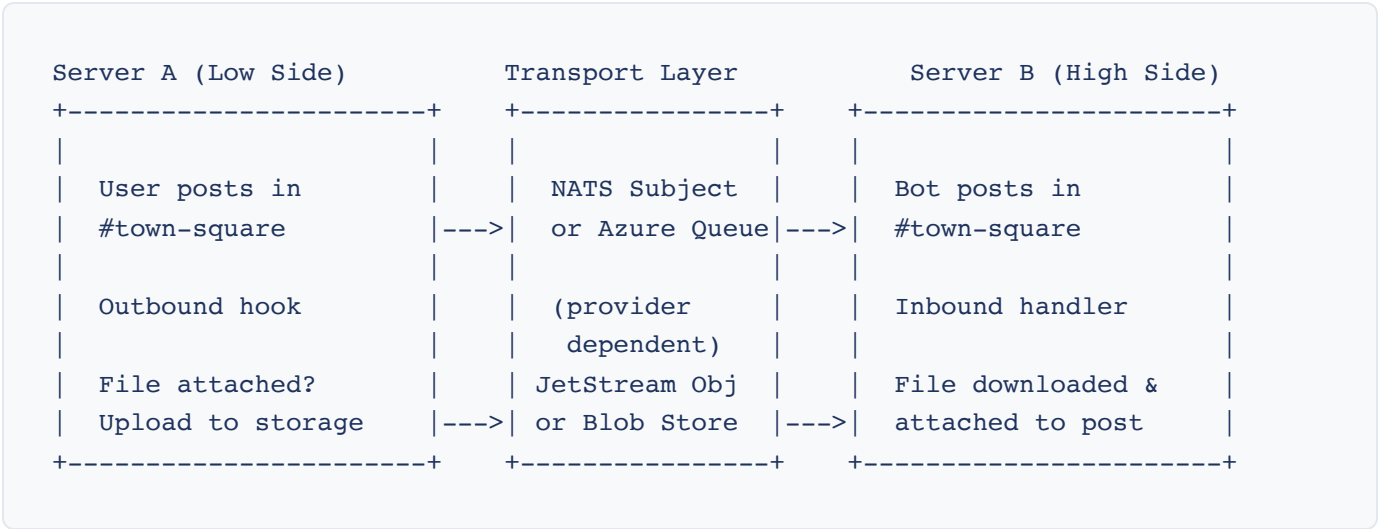
What is Cross Guard?

Cross Guard enables message relay between separate Mattermost servers using NATS or Azure Queue Storage as the transport layer. It is designed for environments where multiple isolated Mattermost instances need to exchange messages across domain boundaries without direct database or API access between servers.

Posts, edits, deletes, reactions, and file attachments flow bidirectionally across server boundaries. Each server maintains its own users, teams, and channels. Cross Guard bridges them at the message level, preserving each server's independence while enabling real-time communication. File attachments are transferred using NATS JetStream Object Store or Azure Blob Storage, depending on the configured provider. File transfer is opt-in per connection.

The plugin integrates with Mattermost's post lifecycle hooks, so relay behavior is transparent to end users. Messages appear in channels as if they were posted locally.

How Federation Works



Two Connection Directions

- **Outbound:** Hooks into Mattermost post lifecycle events (`MessageHasBeenPosted` , `MessageHasBeenUpdated` , `MessageHasBeenDeleted` , reaction events), serializes to a message envelope (JSON or XML, per connection config), and sends via the configured transport.
- **Inbound:** Receives messages from the transport (NATS subscription or Azure queue polling), auto-detects the message format (JSON or XML), deserializes the envelope, and creates, updates, or deletes posts locally.

What Gets Relayed

Event	Relayed?	Notes
New posts	Yes	Including thread replies (root_id preserved)
Post edits	Yes	Updated content replaces original on the remote side
Post deletes	Yes	Matched by original post ID
Reaction adds	Yes	Emoji name and user are relayed. Sync-user reactions are filtered.
Reaction removes	Yes	Matched by emoji name and post ID. Sync-user reactions are filtered.
File attachments	Configurable	Enabled per connection via <code>file_transfer_enabled</code> . Supports allow/deny extension filters.
System messages	No	Filtered out to avoid noise
Relay bot posts	No	Loop prevention via <code>crossguard_relayed</code> property (posts/edits), sync-user filtering (reactions), and delete flag mechanism (deletes)

Key Concepts

Connections

Named connection configurations (NATS or Azure Queue Storage), inbound or outbound, defined in the System Console. Referenced by name in all commands using the format

`direction:name` (e.g., `outbound:relay-to-b`).

Team Initialization

A team must be linked to a connection before any channels in that team can relay messages. This is always the first step. Use `/crossguard init-team` to link a connection.

Channel Initialization

Individual channels within an initialized team are linked to connections. Only linked channels relay messages. Use `/crossguard init-channel` to link a channel.

Inbound Prompts

When an inbound message arrives for a team or channel that hasn't been explicitly accepted, an interactive prompt (Accept/Block buttons) is posted. Team-level prompts appear in town-square; channel-level prompts appear in the target channel. Admins must accept before messages flow. Use `/crossguard reset-prompt` to clear a blocked prompt.

Team Name Rewriting

Maps remote team names to local teams, allowing servers with different team names to relay correctly. Configured per inbound connection using `/crossguard rewrite-team` .

Username Lookup

When enabled (the `UsernameLookup` setting), inbound messages are posted as the matching local user if one exists with the same username. Otherwise, a synthetic relay bot posts them.

Getting Started

- 1 Configure connections** in **System Console > Plugins > Cross Guard**. Add connection details for your chosen provider (NATS or Azure Queue Storage). See the [Admin Configuration Guide](#) for details.
- 2 Initialize a team:** Run `/crossguard init-team [connection-name]` in the team you want to enable.

- 3 **Initialize channels:** Run `/crossguard init-channel [connection-name]` in each channel that should relay messages.
- 4 **Verify:** Run `/crossguard status` to confirm the setup and check that connections are active.

Permission Model

Role	Scope	Notes
SYSTEM ADMIN	All commands, all teams/channels. Can configure connections in System Console. Can test connections via API. Sees global status overview.	Always has full access regardless of <code>RestrictToSystemAdmins</code> setting.
TEAM ADMIN	<code>init-team</code> , <code>teardown-team</code> , <code>init-channel</code> , <code>teardown-channel</code> , <code>reset-prompt</code> , <code>reset-channel-prompt</code> , <code>rewrite-team</code> , <code>status</code> within their team.	Access blocked when <code>RestrictToSystemAdmins</code> is enabled, unless <code>AllowTeamAdminRequests</code> is also enabled (see Request Mode below).
CHANNEL ADMIN	<code>init-channel</code> , <code>teardown-channel</code> within their channel (team must already be initialized).	Access blocked when <code>RestrictToSystemAdmins</code> is enabled, unless <code>AllowChannelAdminRequests</code> is also enabled (see Channel Request Mode below).
MEMBER	<code>status</code> (team-scoped view), <code>help</code> .	Can see which connections are active for their team/channel.

Note

When `RestrictToSystemAdmins` is enabled in plugin settings, Team Admins and Channel Admins lose access to connection management commands. Only System Admins can manage connections.

Request Mode (Team)

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins regain limited access: `init-team` submits a link request for System Admin approval (instead of linking directly), and `teardown-team` allows direct unlinking without approval. System Admins receive request DMs with interactive Approve/Deny buttons. See the [Admin Configuration](#) page for details.

Channel Request Mode

When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins regain limited access: `init-channel` submits a channel link request for Team Admin approval (instead of linking directly), and `teardown-channel` allows direct unlinking without approval. Team Admins receive request DMs with interactive Approve/Deny buttons. See the [Admin Configuration](#) page for details.

Documentation Pages

Slash Command Reference

Complete documentation for all 9 slash commands, including arguments, permissions, and examples.

Admin Configuration Guide

Connection setup for NATS and Azure, relay settings, example configurations, and troubleshooting.

REST API Reference

All 16 REST endpoints with request/response examples and authentication details.

Error Codes

Every numeric `error_code` emitted in Cross Guard logs, grouped by source file, with descriptions and troubleshooting steps.

Transport Interface

Wire protocol reference: message envelope, payload schemas, connection configuration for NATS and Azure Queue Storage, and complete examples.

Threat Model

STRIDE threat analysis, trust zones, data flow documentation, and CDS deployment recommendations.

CDS Whitepaper

Comprehensive architecture, deployment topologies, security analysis, and cross-domain evaluation guide.

Slash Command Reference

All commands for managing Cross Guard relay connections

Overview

- All commands use the trigger `/crossguard`
- Autocomplete is enabled (hints appear as you type)
- Commands return ephemeral responses (only visible to the user who ran them)
- When multiple connections exist and no name is specified, an interactive selection dialog appears

Quick Reference

Command	Description	Permission	Arguments
<code>init-team [name]</code>	Link connection to team	TEAM ADMIN+	Optional connection name
<code>init-channel [name]</code>	Link connection to channel	CHANNEL ADMIN+	Optional connection name
<code>teardown-team [name]</code>	Unlink connection from team	TEAM ADMIN+	Optional connection name
<code>teardown-channel [name]</code>	Unlink connection from channel	CHANNEL ADMIN+	Optional connection name
<code>reset-prompt <name></code>	Clear team connection prompt	TEAM ADMIN+	Required connection name
<code>reset-channel-prompt <name></code>	Clear channel connection prompt	TEAM ADMIN+	Required connection name

Command	Description	Permission	Arguments
<code>rewrite-team [name] [team]</code>	Set/clear team name rewrite	TEAM ADMIN+	See details
<code>status</code>	Show relay status	ANY MEMBER	None
<code>help</code>	Show help	ANY MEMBER	None

/crossguard init-team [connection-name]

Link a connection to the current team, enabling cross-domain relay for this team.

Permission	Team Admin or System Admin (blocked by <code>RestrictToSystemAdmins</code> ; see Request Mode below)
Arguments	<code>connection-name</code> (optional). The display key of the connection (e.g., <code>outbound:relay-to-b</code>).

Behavior

- If omitted and exactly one connection exists in config, auto-selects it
- If omitted and multiple connections exist, opens an interactive selection dialog with all configured connections
- If specified but not found, returns error listing available connections
- If connection already linked to this team, returns "already linked" message
- On first connection linked to team, adds team to initialized teams list
- Posts announcement in `#town-square` : "Cross Guard connection `outbound:relay-to-b` linked to this team by @username"

Request Mode

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins who run this command submit a **link request** instead of linking directly. All System Admins receive a DM with Approve/Deny buttons. The requester is notified of the outcome. If a request is already pending for the same team and connection, the command returns an error.

Examples

```
/crossguard init-team  
/crossguard init-team outbound:relay-to-b
```

Error Cases

- No connections configured. Add connections in the System Console first.
- Connection not found: xyz. Available connections: outbound:relay-to-b, inbound:relay-from-a
- a request is already pending for this connection (request mode only)

/crossguard init-channel [connection-name]

Link a connection to the current channel. The team must be initialized first with the same connection.

Permission	Channel Admin, Team Admin, or System Admin (blocked by <code>RestrictToSystemAdmins</code> ; see Channel Request Mode below)
Arguments	<code>connection-name</code> (optional, same auto-select/dialog behavior as <code>init-team</code>)

Behavior

- Validates team is initialized first. If the team does not yet have this specific connection linked, the REST API auto-links the team. The slash command requires explicit team initialization via `/crossguard init-team`.
- If connection not linked at team level, returns: `Connection X is not linked to this team. Run /crossguard init-team first.`
- Adds link emoji prefix to channel header (visible indicator)
- Publishes WebSocket event `channel_connections_updated` to update UI indicators
- Posts announcement in the channel

Channel Request Mode

When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can run this command to submit a channel connection link request. The request is sent as a

DM to all Team Admins with **Approve** and **Deny** buttons. The channel is not linked until a Team Admin approves the request. Team Admins and System Admins still link channels directly.

Examples

```
/crossguard init-channel  
/crossguard init-channel outbound:relay-to-b
```

/crossguard teardown-team [connection-name]

Unlink a connection from the current team.

Permission	Team Admin or System Admin (blocked by <code>RestrictToSystemAdmins</code> ; see note below)
-------------------	--

Arguments	<code>connection-name</code> (optional). If omitted and multiple connections linked, opens dialog. If one linked, auto-selects.
------------------	---

Behavior

- If no connections linked: `No connections are linked to this team.`
- If this was the last connection, removes team from initialized teams list
- Posts announcement in `#town-square`
- If a pending connection link request exists for the same team and connection, the request is automatically cancelled and the requester is notified

Request Mode

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can run this command to unlink connections directly (no approval needed for unlinking). This is the only connection management action Team Admins can perform without System Admin approval in request mode.

Warning

This command does NOT automatically teardown channel connections. Channels that still have this connection linked become orphaned. Run `/crossguard teardown-channel` in affected channels

first.

Example

```
/crossguard teardown-team outbound:relay-to-b
```

/crossguard teardown-channel [connection-name]

Unlink a connection from the current channel.

Permission	Channel Admin, Team Admin, or System Admin (blocked by <code>RestrictToSystemAdmins</code> ; Channel Admins allowed in channel request mode)
Arguments	<code>connection-name</code> (optional, same auto-select/dialog behavior)

Behavior

- If no connections linked: `No connections are linked to this channel.`
- If this was the last connection on the channel, removes link emoji prefix from channel header
- Publishes WebSocket event to update UI indicators
- Posts announcement: "All relays for this channel are now inactive."
- If a pending channel connection request exists for the same channel and connection, the request is automatically cancelled and the requester is notified

Channel Request Mode

When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can run this command to unlink channel connections directly (no approval needed for unlinking). This is the only channel connection management action Channel Admins can perform without Team Admin approval in channel request mode.

/crossguard reset-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current team. After clearing, a new prompt will appear on the next inbound message.

Permission	Team Admin or System Admin
Arguments	<code>connection-name</code> (required)

Behavior

- Looks up prompt in KV store for this team + connection name
- If no prompt found: `No prompt found for connection X on this team.`
- Deletes the prompt entry, allowing the system to create a fresh prompt on the next inbound message

Use Case

An admin accidentally blocked an inbound connection and wants to re-evaluate it.

Example

```
/crossguard reset-prompt relay-from-a
```

/crossguard reset-channel-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current channel. Same behavior as `reset-prompt` but operates on channel-level prompts.

Permission	Team Admin or System Admin
Arguments	<code>connection-name</code> (required)

Example

```
/crossguard reset-channel-prompt relay-from-a
```

/crossguard rewrite-team [connection-name] [remote-team-name]

Set or clear a remote team name rewrite for an inbound connection. When set, inbound messages with the specified remote team name are routed to this team instead of requiring a matching local team name.

Permission	Team Admin or System Admin
Arguments	<code>connection-name</code> (required if multiple inbound connections linked, optional if exactly one). <code>remote-team-name</code> (optional; omit to clear the rewrite).

Behavior

- Only works with inbound connections. If no inbound connections linked: `No inbound connections are linked to this team.`
- Sets a rewrite index in the KV store: `connection-name + remote-team-name -> local team ID`
- Returns HTTP 409 (Conflict) if the rewrite index already exists for another team
- Posts audit message in `#town-square` showing who made the change
- When clearing: removes the rewrite index entry and clears `RemoteTeamName` from the team connection metadata

Examples

```
/crossguard rewrite-team relay-from-a remote-team-alpha  
/crossguard rewrite-team relay-from-a
```

/crossguard status

Show Cross Guard initialization status for the current team and channel.

Permission	Any team member
Arguments	None

Output Varies by Role

- **Regular users / Team Admins:** Current channel status (relay enabled yes/no), current team status (initialized yes/no), team connections list, channel connections list. Each connection displays file transfer status (e.g., `files off`, `files on`, `files on, allow: .pdf,.docx`).
- **System Admins:** Global overview with all initialized teams (table with display name, team ID, team name, linked connections), all configured connections (table with name, direction, address, auth type, subject, file transfer status), plus current channel status. Warnings shown if connection configuration cannot be parsed.

/crossguard help

Show inline help for all commands. No arguments or special permissions required.

Connection Selection Dialog

When a command needs a connection name and multiple connections are available, an interactive dialog opens automatically.

- Dialog title varies: "Link Connection to Team", "Unlink Connection from Team", etc.
- Dropdown shows all available connections in `direction:name` format
- Dialog submits to `/plugins/crossguard/api/v1/dialog/select-connection`
- State encodes the action and target: `init-team:{teamID}`, `init-channel:{channelID}`, etc.

Admin Configuration Guide

Transport connections, relay settings, and deployment examples

Accessing Plugin Settings

Navigate to **System Console > Plugins > Cross Guard**. The plugin must be enabled before configuration is accessible. Settings are organized into two sections: Connection Configuration and Relay Settings.

Connection Configuration

Understanding Inbound vs Outbound

Direction	Flow	Use When
Outbound	Mattermost → Transport	Your server is the sender . Publishes messages to the configured transport.
Inbound	Transport → Mattermost	Your server is the receiver . Subscribes to or polls messages from the configured transport.

A server can have both inbound and outbound connections simultaneously for bidirectional relay. Multiple connections of the same direction are supported for multi-server topologies.

Provider Selection

Each connection uses either **NATS** or **Azure Queue Storage** as its transport provider. The provider is selected when creating a connection, and the admin console shows only the fields relevant to the chosen provider.

Provider	Transport	File Transfer	Delivery
NATS	Pub/sub messaging via NATS Core	JetStream Object Store	At-most-once
Azure Queue Storage	Polling-based queue (5s interval)	Azure Blob Storage	At-least-once
Azure Blob Storage	Write-ahead log batched to blobs (periodic flush)	Deferred file blobs alongside batch WAL blobs	At-least-once

Common Connection Fields

Field	JSON Key	Required	Validation	Description
Name	<code>name</code>	Yes	Lowercase letters, numbers, and hyphens only. Pattern: <code>^[a-z0-9]+(-[a-z0-9]+)*\$</code> . Must be unique within its direction (inbound or outbound). The same name may appear in both an inbound and an outbound entry, which is the loopback configuration.	Human-readable identifier. Used in all commands and API calls. Examples: <code>relay-to-b</code> , <code>high-side</code> , <code>nats-prod</code>
Provider	<code>provider</code>	Yes	<code>"nats"</code> , <code>"azure-queue"</code> , or <code>"azure-</code>	Transport provider for this connection. Determines which provider-specific

Field	JSON Key	Required	Validation	Description
			blob" . Empty treated as "nats" .	fields are required. Default: "nats" .
File Transfer Enabled	file_transfer_enabled	No	Boolean	Enable file attachment relay for this connection. Default: false .
File Filter Mode	file_filter_mode	No	"" , "allow" , or "deny"	Controls whether file extensions are filtered. allow = only listed extensions are relayed. deny = listed extensions are blocked. Empty = all files pass (default).
File Filter Types	file_filter_types	Conditional	Comma-separated extensions. Case-insensitive, leading dot optional.	List of file extensions used by the filter (e.g. .pdf, .docx, .png). Required when file_filter_mode is allow or deny .
Message Format	message_format	No	"json" or "xml" . Empty treated as "json" .	Wire format for outbound messages. Use "xml" when sending through a Cross Domain Solution (CDS) that requires XML. Inbound messages are auto-detected regardless of this setting. Only applies to outbound connections. Default: "json" .

NATS Provider Fields

These fields appear when the provider is set to **NATS**. In JSON, they are nested under the `"nats"` key (e.g., `"nats": {"address": "...", "subject": "..."}`).

Field	JSON Key	Required	Validation	Description
Address	<code>address</code>	Yes	Non-empty string	NATS server URL. Format: <code>nats://hostname:port</code> . Default port is 4222. Examples: <code>nats://nats.example.com:4222</code> <code>nats://10.0.1.50:4222</code>
Subject	<code>subject</code>	Yes	Must start with <code>crossguard.</code> prefix	NATS subject to publish/subscribe on. Both sides MUST use the same subject. Examples: <code>crossguard.relay.messages</code> , <code>crossguard.prod.sync</code>
TLS Enabled	<code>tls_enabled</code>	No	Boolean	Enable TLS encryption. Required in production with TLS-enabled NATS servers.
Auth Type	<code>auth_type</code>	No	<code>none</code> , <code>token</code> , or <code>credentials</code> . Empty treated as <code>none</code> .	Authentication method for the NATS server.
Token	<code>token</code>	Conditional	Required when <code>auth_type</code> is <code>token</code>	Authentication token for token-based auth.
Username	<code>username</code>	Conditional	Required when <code>auth_type</code> is <code>credentials</code>	Username for credential-based auth.
Password	<code>password</code>	Conditional	Required when <code>auth_type</code> is <code>credentials</code>	Password for credential-based auth

Field	JSON Key	Required	Validation	Description
Client Certificate	<code>client_cert</code>	No	If provided, <code>client_key</code> must also be provided	Path to client TLS certificate file (requires <code>tls_enabled</code>).
Client Key	<code>client_key</code>	No	If provided, <code>client_cert</code> must also be provided	Path to client TLS private key file.
CA Certificate	<code>ca_cert</code>	No	-	Path to CA certificate for verifying the NATS server's TLS certificate.

Azure Queue Provider Fields

These fields appear when the provider is set to **Azure Queue Storage**. In JSON, they are nested under the `"azure_queue"` key (e.g., `"azure_queue": {"queue_service_url": "...", "account_name": "...", "account_key": "...", "queue_name": "..."}`).

Field	JSON Key	Required	Validation	Description
Queue Service URL	<code>queue_service_url</code>	Yes	Non-empty, valid URL	Azure Queue Azure, the storage URL is <code>https://<account_name>.queue.azure.com/</code> . For Azurite (local emulator), the URL is <code>http://localhost:10000/</code> .
Blob Service URL	<code>blob_service_url</code>	Conditional	Required when <code>file_transfer_enabled</code> is <code>true</code> . Must be a valid URL.	Azure Blob Storage Azure, the storage URL is <code>https://<account_name>.blob.azure.com/</code> .
Account Name	<code>account_name</code>	Yes	Non-empty string	Azure Storage account name.
Account Key	<code>account_key</code>	Yes	Non-empty string	Azure Storage account key (base64). The key is displayed in the Azure Portal under the account's Access Keys tab.

Field	JSON Key	Required	Validation	Description
Queue Name	queue_name	Yes	Non-empty string	Name of the queue. If the queue already exists, use the same queue name.
Blob Container Name	blob_container_name	Conditional	Required when file_transfer_enabled is true	Name of the blob container. If the container already exists, use the same container name.

Azure Blob Provider Fields

These fields appear when the provider is set to **Azure Blob Storage** (the batched WAL transport, distinct from the Azure Queue provider above). In JSON, they are nested under the "azure_blob" key. Instead of a queue, the outbound side batches messages into a write-ahead log blob and flushes it on an interval; the inbound side polls the container, takes a short-lived lease on each batch blob, processes it, and deletes it. File attachments are stored as deferred blobs alongside the batch blobs in the same container.

Field	JSON Key	Required	Validation	Description
Service URL	service_url	Yes	Non-empty, valid URL	Azure Blob Storage service URL. For Azure, the standard form is https://<account>.blob.core.windows.net/<container>. For Azurite, use http://azurite:10000/<container>.
Account Name	account_name	Yes	Non-empty string	Azure Storage account name. Combined with account_key to form a connection string.
Account Key	account_key	Yes	Non-empty string	Azure Storage account secret key (base64). Treat as a secret.
Blob Container Name	blob_container_name	Yes	Non-empty string	Name of the Azure Blob container. If the container already exists, use the same container name. If it does not exist, both sides will create it.

Field	JSON Key	Required	Validation	Description
Flush Interval	flush_interval_seconds	No	Integer. If set, must be at least 5 .	How often the outbound write-ahead log blob and seconds. Lower values reduce latency at the cost of more
Blob Lock Max Age	blob_lock_max_age_seconds	No	Integer. If set, must be at least 30 .	Maximum age of a blob lock side considers it stale and Default: 300 seconds (5 minutes) crashed inbound node from

Azure Service Bus Provider Fields

These fields appear when the provider is set to **Azure Service Bus**. In JSON, they are nested under the `"azure_servicebus"` key. Service Bus uses an AMQP-based queue for message relay with PeekLock semantics: the plugin completes messages on successful handler invocation and abandons them on handler error (Service Bus then increments delivery count and routes to the dead-letter queue after the queue-configured `MaxDeliveryCount`).

Important: The plugin does *not* auto-create the queue. Pre-create it in the Azure Portal or via ARM/Terraform with your chosen `LockDuration` (default 60 s, maximum 5 min) and `MaxDeliveryCount` before saving the connection. Lock duration is an entity property; the plugin cannot configure it.

Why the blob fields are named differently from the Azure Queue provider: Service Bus and Azure Storage are separate Azure resources with independent credentials. The Azure Queue provider shares one storage account for both queues and blobs; the Service Bus provider needs a distinct credential pair for its Blob sidecar (which is why you see `blob_account_name` / `blob_account_key` here instead of the unqualified `account_name` / `account_key`).

Field	JSON Key	Required	Validation	D
Connection String	connection_string	Yes	Non-empty string	S I < re

Field	JSON Key	Required	Validation	D
Queue Name	queue_name	Yes	Up to 260 characters; letters, digits, periods, hyphens, underscores, and forward slashes.	N
Blob Service URL	blob_service_url	Only when file_transfer_enabled is true	Valid URL	A
Blob Account Name	blob_account_name	Only when file_transfer_enabled is true	Non-empty string	S
Blob Account Key	blob_account_key	Only when file_transfer_enabled is true	Non-empty string	S
Blob Container Name	blob_container_name	Only when file_transfer_enabled is true	Non-empty string	B
Max Message Size	max_message_size_bytes	No	Integer, 1024 to 104857600. Default 192000 .	M
Blob Poll Interval	blob_poll_interval_seconds	No	Integer, ≥ 1. Default 15 .	Ir

Dead-Letter Queue (DLQ) behaviour: If handler errors cause redelivery count to exceed the queue's `MaxDeliveryCount` , Azure Service Bus auto-moves the message to the queue's dead-letter sub-queue. The plugin does NOT inspect or process DLQ messages; they stop appearing in receive polls. Use the Azure Portal, `az servicebus queue message` CLI, or Azure monitoring

alerts to inspect the DLQ. The plugin logs a WARN with error code `26004` (`ServiceBusRedelivery`) on every redelivery so operators can detect poison-message buildup before DLQ moves happen.

Azure Service Principal Authentication

All three Azure providers (Queue, Blob, Service Bus) support **Azure AD Service Principal authentication** as an alternative to shared keys / SAS connection strings. Service Principal uses the AAD `client_credentials` grant: an application registered in Azure AD authenticates with a tenant ID, a client (application) ID, and a client secret. The plugin acquires bearer tokens from AAD and uses them to call the Azure data plane.

Why this matters: Shared keys grant full account access and cannot be scoped or rotated centrally. Service Principal + Azure RBAC lets you grant a single AAD identity exactly the permissions it needs on a specific queue, container, or namespace, and rotation happens in AAD without touching plugin config.

Phase 1 Deployment Guidance

This release supports the **Client Secret** flow only. Client certificate auth and Managed Identity are tracked as later phases. From a compliance / threat-model perspective:

- **Acceptable for:** development, staging, and production environments paired with a strict secret-rotation policy (90-day maximum). Federal customers using Azure Government with rotation tooling can deploy Phase 1 in production.
- **Not the recommended posture for:** high-side ATO environments where shared-secret material at rest is a finding. For those environments, wait for Phase 2 (certificate auth via Azure Key Vault + CSI driver) or use the Mattermost env-var substitution mechanism described below to keep the secret out of plugin config entirely.

Known Exposure: Secret in Plugin Config (Phase 1B Follow-Up)

The Mattermost `GET /api/v4/config` endpoint returns plugin custom-settings as cleartext to any System Admin. This affects **all** Crossguard secret fields (`account_key` , `connection_string` , `blob_account_key` , `client_secret`): an admin opening the System Console will see the secret value in their browser's network tab and any `mmctl config show` output will include it. A structural fix that replaces the cleartext with a reference is tracked as Phase 1B.

Immediate Federal mitigation: inject the entire connections JSON via Mattermost's plugin-settings env-var substitution. Setting `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` (and the inbound counterpart) overrides whatever is stored in plugin config at load time; the value can come from a Kubernetes Secret, a CSI volume + init container, systemd `LoadCredential=`, or HashiCorp Vault. The credential then lives only in the Mattermost process environment, not in `config.json` on disk, not in the configuration database, and not in `GET /api/v4/config` responses. This is the same mechanism Mattermost uses for every other plugin secret.

Common Service Principal Fields

The following fields are shared across all three Azure providers when `auth_mode` is `service-principal`:

Field	Required	Type	Description
<code>auth_mode</code>	No	String enum. Queue/Blob: <code>"shared-key"</code> (default) or <code>"service-principal"</code> . Service Bus: <code>"connection-string"</code> (default) or <code>"service-principal"</code> .	Selects the credential type. Empty default for backward compatibility (existing configs).
<code>azure_cloud</code>	No	String enum: <code>"public"</code> (default), <code>"usgov"</code> , <code>"china"</code> .	Selects the Azure cloud environment. Public host for credential acquisition. For Queue the data-plane SDK client options. For Service Bus routing comes from the FQDN namespace receives the cloud option.
<code>tenant_id</code>	Yes (SP)	String. GUID or domain form (e.g., <code>contoso.onmicrosoft.com</code>).	Azure AD directory (tenant) ID.
<code>client_id</code>	Yes (SP)	String (GUID).	Azure AD application (client) ID for the service principal.
<code>client_secret</code>	Yes (SP)	String (secret).	Inline client secret value. Required in SP mode. Inject the secret into the <code>MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS</code> env var (see Plugin Settings).

Azure Service Bus – Additional SP Field

Field	Required	Type	Description
<code>service_bus_namespace</code>	Yes (SP)	String (FQDN).	Fully-qualified Service Bus namespace, e.g., <code>myns.servicebus.windows.net</code> for Public, <code>myns.servicebus.usgovcloudapi.net</code> for US Government. The SAS connection string used to encode this for the legacy mode; in SP mode it must be supplied separately. If you accidentally include the <code>sb://</code> scheme, it is stripped.

Minimum Required Azure RBAC Roles

Grant the Service Principal these data-plane roles at the resource scope. The plugin **skips auto-creation** of queues/containers in SP mode — you must pre-provision them via the Azure Portal, ARM template, or `az` CLI. This means the operator does not need management-plane (Owner / Contributor) roles, which is the correct least-privilege posture.

- **Azure Queue provider:** `Storage Queue Data Contributor` on the Queue Storage account. If file transfer is enabled, also grant `Storage Blob Data Contributor` on the same account.
- **Azure Blob provider:** `Storage Blob Data Contributor` on the Blob Storage account.
- **Azure Service Bus provider:** `Azure Service Bus Data Sender` AND `Azure Service Bus Data Receiver` on the queue (or on the namespace). If file transfer is enabled and the blob sidecar inherits the parent SP credential (the default in SP mode), the same SP also needs `Storage Blob Data Contributor` on the blob storage account.

Owner-tier roles are NOT required. If you see startup errors about

`AuthorizationPermissionMismatch` on a Create call, double-check that the queue/container has been pre-provisioned and that the SP has the correct data-plane role.

Service Bus Blob Sidecar Credential Inheritance (SP Mode)

When Azure Service Bus is in SP mode the blob sidecar **inherits the parent SP credential**: do NOT set `blob_account_name` or `blob_account_key`, validation rejects the save if you do. This empty-fields rule applies unconditionally in SP mode, even when `file_transfer_enabled` is false, so a stored shared-key cannot linger across a file-transfer toggle. When file transfer is enabled, provide `blob_service_url` and `blob_container_name`. The same Service Principal must have the appropriate roles on both the Service Bus namespace and the blob storage account. Mixed-credential deployments (one SP for AMQP, a different SP or shared key for blob) are deferred to Phase 2.

Azure Government (Sovereign Cloud) Setup

To configure a connection for Azure US Government:

1. Set `azure_cloud` to `usgov`.
2. Use the US Gov endpoints in `queue_service_url`, `blob_service_url`, `service_url`, or `service_bus_namespace`: `*.queue.core.usgovcloudapi.net`, `*.blob.core.usgovcloudapi.net`, `*.servicebus.usgovcloudapi.net`.
3. Register the Service Principal in the Azure US Government AAD tenant (not the commercial tenant). Grant the data-plane RBAC roles described above.

The plugin passes the cloud configuration to `azidentity` so token acquisition routes to `login.microsoftonline.us` automatically; you do not need to override an authority host.

Save-Time Credential Probe

When you save a plugin configuration that includes one or more SP connections, the plugin acquires a token from AAD for each SP connection before the save commits. Bad tenant/client/secret values fail the save with a sanitized error message instead of producing repeated runtime failures during the relay loop. The probe has a 10-second timeout. Errors are scrubbed of any credential material before being logged or surfaced to the admin UI.

Secret Hydration in the Admin Console

When you open an existing connection in the System Console, password fields (including `client_secret`, `account_key`, `connection_string`, `blob_account_key`) are hydrated with a distinctive sentinel token (`__CROSSGUARD_SECRET_UNCHANGED__`) instead of the stored cleartext. The token is rendered as bullets in the password input and never round-trips to a browser as a real secret. If you save the form without editing a password field, the sentinel is sent back to the server and the stored value is preserved.

- **To rotate a secret:** paste the new value into the field and save.
- **To leave a secret unchanged:** do nothing — do not delete the field contents.
- **To clear a stored secret** (for example, when migrating from shared-key to service-principal): clear the field and save. An empty inbound value is honored as an explicit clear.

Validation Rules Summary

1. Names must be unique within each direction (inbound or outbound). The same bare name may appear in both lists, which is the loopback configuration where one server is both

publishing and subscribing on the same transport

2. Names must be lowercase alphanumeric with hyphens (no spaces, underscores, uppercase)
3. Subject must always start with `crossguard.`
4. Both sides of a relay must use the same NATS subject
5. Auth credentials must be complete (both username AND password for credentials auth)
6. TLS cert and key must be provided as a pair (enforced at config save time)
7. Azure Queue connections require `queue_service_url` and `queue_name` ; shared-key mode additionally requires `account_name` and `account_key`
8. Azure Queue connections require `blob_service_url` and `blob_container_name` when file transfer is enabled
9. Azure Blob connections require `service_url` and `blob_container_name` ; shared-key mode additionally requires `account_name` and `account_key`
10. Azure Blob `flush_interval_seconds` must be at least 5 when set;
`blob_lock_max_age_seconds` must be at least 30 when set
11. Azure Service Bus connections require `queue_name` ; connection-string mode additionally requires `connection_string` ; service-principal mode additionally requires `service_bus_namespace`
12. Azure Service Bus `queue_name` is capped at 260 characters and allows only `A-Z a-z 0-9 . _ - /`
13. Azure Service Bus connections with file transfer enabled also require `blob_service_url` and `blob_container_name` ; in connection-string mode also `blob_account_name` and `blob_account_key` . In service-principal mode `blob_account_name` and `blob_account_key` must be **empty** (sidecar inherits parent SP); this empty-fields rule applies unconditionally in SP mode, including when file transfer is disabled, so stale shared-key material cannot survive a file-transfer toggle
14. When `auth_mode` is `service-principal` : `tenant_id` (GUID or FQDN), `client_id` , and `client_secret` are required; the corresponding legacy field (`account_key` / `connection_string`) must be empty
15. When `auth_mode` is set, `azure_cloud` must be one of `public` , `usgov` , or `china` when present; empty defaults to `public`

Relay Settings

Username Lookup

Setting	UsernameLookup
Type	Boolean
Default	true

When enabled: Inbound messages look for a local user with the same username as the remote sender. If found, the message is posted as that local user. This creates a seamless experience where messages appear to come from the correct person.

When disabled: All inbound messages are posted by the Cross Guard relay bot, with the remote username shown in the message formatting. Use this if usernames don't match across servers.

Restrict to System Admins

Setting	RestrictToSystemAdmins
Type	Boolean
Default	false

When enabled: Only System Admins can run `init-team`, `teardown-team`, `init-channel`, `teardown-channel`, `reset-prompt`, `reset-channel-prompt`, and `rewrite-team`. Team Admins and Channel Admins are denied access.

When disabled: Team Admins can manage team connections, Channel Admins can manage channel connections (the default permission hierarchy).

Tip

Enable this setting in high-security environments where only centralized admins should control cross-domain relay. Pair it with `AllowTeamAdminRequests` below to let Team Admins request connections without granting them direct control.

Allow Team Admin Requests

Setting	<code>AllowTeamAdminRequests</code>
Type	Boolean
Default	<code>false</code>
Requires	<code>RestrictToSystemAdmins</code> must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Team Admins can submit connection link requests via `/crossguard init-team` or the REST API. The request is sent as a direct message to every System Admin with interactive **Approve** and **Deny** buttons. Team Admins can also unlink connections directly with `/crossguard teardown-team` (no approval needed for unlinking).

When disabled: Team Admins are fully blocked from connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Team Admins already have direct access.

Request workflow

When a Team Admin submits a link request, all System Admins receive a DM from the Cross Guard bot with the requester's name, team, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

Allow Channel Admin Requests

Setting	<code>AllowChannelAdminRequests</code>
Type	Boolean
Default	<code>false</code>
Requires	<code>RestrictToSystemAdmins</code> must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Channel Admins can submit channel connection link requests via `/crossguard init-channel` or the REST API. The request

is sent as a direct message to every Team Admin with interactive **Approve** and **Deny** buttons. Channel Admins can also unlink channel connections directly with `/crossguard teardown-channel` (no approval needed for unlinking).

When disabled: Channel Admins are fully blocked from channel connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Channel Admins already have direct access.

Request workflow

When a Channel Admin submits a channel link request, all Team Admins for that team receive a DM from the Cross Guard bot with the requester's name, team, channel, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the channel link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

File Transfer Settings

File transfer is configured per connection, not globally. Each connection can independently enable or disable file attachment relay and apply extension filters.

Requirements

NATS Provider

- **JetStream-enabled NATS server:** File transfer uses NATS JetStream Object Store. Core NATS alone is not sufficient. The NATS server must have JetStream enabled.
- **Object Store bucket:** The plugin auto-creates a `crossguard-files` bucket with a 1-hour TTL on first use. No manual NATS configuration is needed.
- **Max file size:** Governed by the Mattermost server's `FileSettings.MaxFileSize` configuration (default 100 MB). Files exceeding this limit are silently skipped with a log warning.

Azure Provider

- **Blob container:** File transfer uses Azure Blob Storage. The plugin auto-creates the container specified in `blob_container_name` if it does not exist.

- **Blob metadata:** Headers (post ID, connection name, filename) are stored as Base64-encoded JSON in the blob's `crossguard_headers` metadata key.
- **Polling interval:** The inbound blob watcher polls every 15 seconds. Files are deleted after successful processing.
- **Max file size:** Same as NATS: governed by the Mattermost server's `FileSettings.MaxFileSize` configuration.

Filter Examples

Goal	file_filter_mode	file_filter_types
Allow all files (default)	<code>""</code>	<code>""</code>
Allow only PDFs and images	<code>"allow"</code>	<code>".pdf,.png,.jpg,.gif"</code>
Block executables	<code>"deny"</code>	<code>".exe,.bat,.sh,.cmd"</code>

Note

For bidirectional file relay, both the outbound and inbound connections must have `file_transfer_enabled` set to `true`. File filters are evaluated independently on each side, so the outbound connection can filter what it sends and the inbound connection can filter what it accepts.

Testing Connections

The admin console UI includes a "Test Connection" button for each configured connection.

- **API endpoint:** `POST /plugins/crossguard/api/v1/test-connection` (System Admin only)
- **NATS outbound test:** Publishes a message with a unique ID and confirms flush
- **NATS inbound test:** Subscribes to the subject and confirms the subscription is active
- **Azure test:** Validates the connection string by enqueueing and deleting a test message
- Tests use a temporary connection (does not affect the running plugin's connections)
- The `direction` query parameter is NATS-specific; Azure tests validate connectivity regardless of direction

Example Configurations

Dual-Server Setup (Server A sends to Server B)

Server A (sender)

```
Outbound Connections:
[
  {
    "name": "relay-to-b",
    "provider": "nats",
    "nats": {
      "address": "nats://nats.example.com:4222",
      "subject": "crossguard.relay.messages",
      "auth_type": "token",
      "token": "shared-secret-token"
    },
    "file_transfer_enabled": true,
    "file_filter_mode": "allow",
    "file_filter_types": ".pdf,.docx,.png"
  }
]

Inbound Connections: []
```

Server B (receiver)

```
Inbound Connections:
[
  {
    "name": "relay-from-a",
    "provider": "nats",
    "nats": {
      "address": "nats://nats.example.com:4222",
      "subject": "crossguard.relay.messages",
      "auth_type": "token",
      "token": "shared-secret-token"
    },
    "file_transfer_enabled": true
  }
]

Outbound Connections: []
```

Bidirectional Setup (Both servers send and receive)

Server A

```
Outbound: [{"name": "to-b", "provider": "nats", "nats": {"address": "nats://nats:4222"}  
Inbound:  [{"name": "from-b", "provider": "nats", "nats": {"address": "nats://nats:4222"}
```

Server B

```
Outbound: [{"name": "to-a", "provider": "nats", "nats": {"address": "nats://nats:4222"}  
Inbound:  [{"name": "from-a", "provider": "nats", "nats": {"address": "nats://nats:4222"}
```

Note

Each direction uses a different subject to avoid echo loops. Server A publishes on `crossguard.a-to-b` and Server B subscribes to it, and vice versa.

Docker Dev Environment

- **Server A:** `http://low.test:8075` (admin/password, usera/password, Team: Test A)
- **Server B:** `http://high.test:8076` (admin/password, userb/password, Team: Test B)
- **NATS:** `nats://nats:4222` (from plugin's perspective inside Docker network)
- **Setup:** `make docker-setup` , **Deploy:** `make deploy`

Azure Queue Storage Setup

Server A (sender via Azure Queue)

```
Outbound Connections:  
[  
  {"name": "azure-to-b",  
   "provider": "azure-queue",  
   "azure_queue": {  
     "queue_service_url": "https://mystorageaccount.queue.core.windows.net",  
     "blob_service_url": "https://mystorageaccount.blob.core.windows.net",  
     "account_name": "mystorageaccount",  
     "account_key": "base64-account-key",  
     "queue_name": "crossguard-relay",  
     "blob_container_name": "crossguard-files"  
   },  
  ],
```

```
"file_transfer_enabled": true
}]
```

Server B (receiver via Azure Queue)

```
Inbound Connections:
[
  {
    "name": "azure-from-a",
    "provider": "azure-queue",
    "azure_queue": {
      "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
      "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
      "account_name": "mystorageaccount",
      "account_key": "base64-account-key",
      "queue_name": "crossguard-relay",
      "blob_container_name": "crossguard-files"
    },
    "file_transfer_enabled": true
  }
]
```

Azure Blob variant (batched WAL transport)

```
Outbound Connections:
[
  {
    "name": "azure-blob-to-b",
    "provider": "azure-blob",
    "azure_blob": {
      "service_url": "https://mystorageaccount.blob.core.windows.net",
      "account_name": "mystorageaccount",
      "account_key": "base64-account-key",
      "blob_container_name": "crossguard-wal",
      "flush_interval_seconds": 60,
      "blob_lock_max_age_seconds": 300
    },
    "file_transfer_enabled": true
  }
]
```

Note

Both sides share the same Azure Storage account, queue name, and blob container. The outbound side enqueues messages and uploads blobs; the inbound side polls and processes them.

Troubleshooting

Problem	Solution
"No connections configured"	Add connections in System Console > Plugins > Cross Guard first.
NATS connection test fails	Verify NATS server is reachable, check address/port, verify auth credentials match.
Messages not relaying	Run <code>/crossguard status</code> . Verify both team AND channel are initialized. Check that transport identifiers match on both sides (NATS subject for NATS connections, queue name for Azure connections).
"Connection not found"	Connection name is case-sensitive and uses <code>direction:name</code> format (e.g., <code>outbound:relay-to-b</code>).
Orphaned connections	If a connection name is changed in config, existing team/channel links reference the old name. Teardown and re-init with the new name.
Inbound messages blocked	Check if an inbound prompt was blocked. Run <code>/crossguard reset-prompt <name></code> to clear it and re-evaluate.
Wrong team receives messages	Check team name rewrite settings with <code>/crossguard status</code> . Remote team names must match or be rewritten.
Files not transferring	Verify <code>file_transfer_enabled</code> is <code>true</code> on both outbound and inbound connections. For NATS, ensure the server has JetStream enabled (not just core NATS). For Azure, verify <code>blob_container_name</code> is set on both outbound and inbound connections.
"File semaphore full" log warnings	Too many concurrent file transfers. The limit is 32 simultaneous file operations. Large files or slow connections can cause congestion. Consider reducing file sizes or upgrading network bandwidth.
File blocked by filter	Check <code>file_filter_mode</code> and <code>file_filter_types</code> on the connection. An <code>allow</code> filter rejects extensions not in the list; a <code>deny</code> filter rejects extensions in the list.

Problem	Solution
File appears after text message	Normal behavior. Files arrive asynchronously via the file watcher (NATS Object Store watcher or Azure Blob poller). The inbound watcher retries post mapping lookup up to 3 times with backoff to wait for the text message to be processed first.
Large files silently dropped	Files larger than the server's <code>MaxFileSize</code> setting (default 100 MB) are skipped with a log warning. Check <code>FileSettings.MaxFileSize</code> in the Mattermost server configuration.
Azure connection test fails	Verify the connection string is correct. Check that the storage account is accessible from the server. Ensure the account key has not been rotated.
Azure messages delayed	Azure Queue Storage uses polling (5-second intervals for messages, 15-second intervals for files). This is normal. For lower latency, use the NATS provider.
Azure message too large	Azure Queue Storage has a 48,000-byte message size limit (before Base64 encoding to the 64 KB wire limit). Messages exceeding this limit have their text truncated to fit. Long posts may lose content.
Duplicate messages with Azure	Azure Queue provides at-least-once delivery. The plugin uses idempotency checks (post ID mapping) to prevent duplicates, but brief visibility timeout races can cause rare duplicates during error recovery.
Azure blob files not appearing	Verify <code>blob_container_name</code> is set on both sides. Check that <code>file_transfer_enabled</code> is <code>true</code> on both outbound and inbound connections. The blob watcher polls every 15 seconds.

REST API Reference

All Cross Guard REST endpoints with request and response examples

Overview

Base Path	/plugins/crossguard/api/v1/
Authentication	All endpoints require the Mattermost-User-Id header (set automatically by Mattermost for authenticated requests).
Request Limit	5 MB max request body
Response Format	JSON
Error Format	{"error": "message"} with appropriate HTTP status code

Connection Testing

POST

/api/v1/test-connection

SYSTEM ADMIN

Test a connection configuration before saving. Supports NATS, Azure Queue, Azure Blob, and Azure Service Bus providers.

Query Parameters

direction	Optional. inbound or outbound (defaults to outbound). NATS-specific; ignored for Azure Queue, Azure Blob, and Azure Service Bus connections.
-----------	---

Request Body (NATS)

```
{
  "name": "relay-to-b",
  "provider": "nats",
  "nats": {
```

```
"address": "nats://nats.example.com:4222",
"subject": "crossguard.relay.messages",
"tls_enabled": false,
"auth_type": "token",
"token": "my-secret-token"
}
}
```

Request Body (Azure Queue)

```
{
  "name": "azure-relay",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay"
  }
}
```

Request Body (Azure Blob)

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal"
  }
}
```

Request Body (Azure Service Bus)

```
{
  "name": "azure-servicebus-relay",
  "provider": "azure-servicebus",
  "azure_servicebus": {
    "connection_string": "Endpoint=sb://myns.servicebus.windows.net/;SharedAccess
    "queue_name": "crossguard-relay"
```

```
}  
}
```

Validation

NATS

- `nats.address` required
- `nats.subject` required and must start with `crossguard.`
- `nats.auth_type` must be `none`, `token`, `credentials`, or empty
- `nats.token` required if `auth_type` is `token`
- `nats.username` + `nats.password` required if `auth_type` is `credentials`

Azure Queue

- `azure_queue.queue_service_url` required
- `azure_queue.queue_name` required
- `azure_queue.account_name` and `azure_queue.account_key` required when `azure_queue.auth_mode` is empty or `shared-key`; **must be empty** when `auth_mode` is `service-principal`
- When `file_transfer_enabled` is true, `blob_service_url` and `blob_container_name` are also required

Azure Blob

- `azure_blob.service_url` required
- `azure_blob.blob_container_name` required
- `azure_blob.account_name` and `azure_blob.account_key` required when `azure_blob.auth_mode` is empty or `shared-key`; **must be empty** when `auth_mode` is `service-principal`

Azure Service Bus

- `azure_servicebus.queue_name` required; at most 260 characters; allowed characters: letters, digits, periods, hyphens, underscores, and forward slashes
- `azure_servicebus.connection_string` required when `auth_mode` is empty or `connection-string`; **must be empty** when `auth_mode` is `service-principal`
- `azure_servicebus.service_bus_namespace` required when `auth_mode` is `service-principal` (FQDN-shaped, e.g., `myns.servicebus.windows.net`; the `sb://` scheme is stripped if present)

- When `file_transfer_enabled` is true, `blob_service_url` and `blob_container_name` are required. In `connection-string` mode, `blob_account_name` and `blob_account_key` are also required; in `service-principal` mode they **must be empty** (the blob sidecar inherits the parent SP credential)
- `azure_servicebus.max_message_size_bytes` (optional) must be between 1024 and 104857600 when set
- `azure_servicebus.blob_poll_interval_seconds` (optional) must be at least 1 when set

Service Principal Auth (all three Azure providers)

When the per-provider `auth_mode` is `service-principal`, the request body must additionally include the shared SP fields (`tenant_id`, `client_id`, `client_secret`, and optionally `azure_cloud`). See the [Azure Service Principal Authentication](#) section in the admin guide for the full field reference and minimum RBAC roles. A 400 response is returned when SP validation fails (bad tenant format, missing `client_id`, missing `client_secret`, or cross-mode field leakage).

Success Response (outbound) - 200

```
{"status": "ok", "message": "Test message sent with ID: abc123", "id": "abc123"}
```

Success Response (inbound) - 200

```
{"status": "ok", "message": "Connected and subscribed to subject successfully"}
```

Success Response (Azure Queue) - 200

```
{"status": "ok", "message": "Azure Queue Storage connection test successful"}
```

Success Response (Azure Blob) - 200

```
{"status": "ok", "message": "Azure Blob Storage connection test successful"}
```

Success Response (Azure Service Bus) - 200


```
{"status": "ok", "message": "Azure Service Bus connection test successful"}
```

Error Responses

Status	Meaning
400	Validation error
401	Not authenticated
403	Not system admin
502	Transport connection failed (NATS unreachable, or Azure credentials/service URL invalid)

Team Management

POST `/api/v1/teams/{team_id}/init` **TEAM ADMIN+**

Link a connection to a team (API equivalent of `/crossguard init-team`).

Path Parameters

team_id 26-character Mattermost team ID

Query Parameters

connection_name Optional. Connection in `direction:name` format (e.g., `outbound:relay-to-b`).

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
}
```

```
"team_name": "test-a",
"connection_name": "outbound:relay-to-b"
}
```

Request Mode Response - 200

When the caller is a Team Admin and both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, the endpoint submits a link request instead of linking directly:

```
{
  "status": "request_submitted",
  "message": "Your request to link connection `outbound:relay-to-b` ...",
  "team_id": "abc123...",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
400	Invalid team_id, no connections configured, or connection not found (includes <code>connections</code> array of available names)
403	Insufficient permissions
404	Team not found
409	A request is already pending for this connection (request mode only)
500	Internal error

POST `/api/v1/teams/{team_id}/teardown` **TEAM ADMIN+**

Unlink a connection from a team. If a pending connection link request exists for the same team and connection, it is automatically cancelled.

Path Parameters

team_id

26-character Mattermost team ID

Query Parameters

connection_name

Optional. Connection to unlink.

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

POST

/api/v1/channels/{channel_id}/init

CHANNEL ADMIN+

Link a connection to a channel (team auto-inits if needed).

Path Parameters

channel_id

26-character Mattermost channel ID

Query Parameters

connection_name

Optional. Connection in **direction:name** format.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
}
```

```
"connection_name": "outbound:relay-to-b"
}
```

Channel Request Mode Response - 200

When `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are both enabled and the caller is a Channel Admin (not a Team Admin or System Admin), the endpoint submits a channel connection request instead of linking directly:

```
{
  "status": "request_submitted",
  "message": "Your request to link connection ... has been submitted for team ad
  "channel_id": "xyz789...",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
400	Invalid input or no connections
403	Insufficient permissions
404	Channel not found
409	A channel request is already pending for this connection
500	Internal error

POST `/api/v1/channels/{channel_id}/teardown`

CHANNEL ADMIN+

Unlink a connection from a channel. If a pending channel connection request exists for the same connection, it is automatically cancelled.

Path Parameters

channel_id

26-character Mattermost channel ID

Query Parameters

connection_name

Optional. Connection to unlink.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Status

GET

/api/v1/status

SYSTEM ADMIN

Global status of all initialized teams and transport connections.

Response - 200

```
{
  "teams": [
    {
      "team_id": "abc123...",
      "team_name": "test-a",
      "display_name": "Test A",
      "linked_connections": [
        {"direction": "outbound", "connection": "relay-to-b"}
      ]
    }
  ],
  "connections": [
    {
      "name": "relay-to-b",

```

```

    "direction": "outbound",
    "provider": "nats",
    "address": "nats://nats:4222",
    "auth_type": "token",
    "subject": "crossguard.relay.messages",
    "file_transfer_enabled": true,
    "file_filter_mode": "allow",
    "file_filter_types": ".pdf,.docx",
    "message_format": "json"
  },
  {
    "name": "azure-relay",
    "direction": "outbound",
    "provider": "azure-queue",
    "queue_name": "crossguard-relay",
    "file_transfer_enabled": false,
    "message_format": "json"
  },
  {
    "name": "azure-blob-relay",
    "direction": "outbound",
    "provider": "azure-blob",
    "blob_container_name": "crossguard-wal",
    "file_transfer_enabled": false,
    "message_format": "json"
  },
  {
    "name": "azure-servicebus-relay",
    "direction": "outbound",
    "provider": "azure-servicebus",
    "queue_name": "crossguard-relay",
    "file_transfer_enabled": false,
    "message_format": "json"
  }
]
}

```

Note

Connection details are redacted. Tokens, passwords, and TLS cert paths are not included.

GET

/api/v1/teams/{team_id}/status

TEAM MEMBER

Initialization and connection status for a specific team.

Response - 200

```
{
  "team_id": "abc123...",
  "team_name": "test-a",
  "team_display_name": "Test A",
  "initialized": true,
  "request_mode": false,
  "linked_connections": [
    {"direction": "outbound", "connection": "relay-to-b"}
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Response Fields

orphaned

true when the connection is linked to the team but no longer exists in the plugin configuration.

remote_team_name

Set when a team name rewrite is configured for this inbound connection.

request_mode

true when the calling user is a Team Admin and the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled). Omitted for System Admins.

request_pending

true when a link request is pending for this unlinked connection. Only populated when the plugin is in request mode.

GET

`/api/v1/channels/{channel_id}/status`

CHANNEL MEMBER

Connection status for a specific channel.

Response - 200

```
{
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "channel_display_name": "Town Square",
  "team_name": "Test A",
  "channel_request_mode": true,
  "team_connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Channel request mode fields

`channel_request_mode` (top-level) is `true` when the caller is a Channel Admin operating in channel request mode (not a Team Admin or System Admin). `request_pending` (per connection) is `true` when an unresolved channel link request exists for that connection. Both fields are omitted when not applicable.

Note

Returns an error for DM and group message channels: "Cross Guard is not available for direct or group messages"

GET**/api/v1/channels/connections****AUTHENTICATED**

Bulk query which channels have active Cross Guard connections.

Query Parameters

ids

Required. Comma-separated channel IDs (max 2048 IDs).

Response - 200

```
{
  "channel_id_1": "outbound:relay-to-b,inbound:relay-from-a",
  "channel_id_2": "outbound:relay-to-b"
}
```

- Only returns channels the user is a member of
- Invalid IDs and channels without connections are omitted
- Used internally by the webapp to show channel indicators

Team Name Rewriting

POST**/api/v1/teams/{team_id}/rewrite****TEAM ADMIN+**

Set a remote team name rewrite for an inbound connection.

Request Body

```
{
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Validation

- Both `connection` and `remote_team_name` must be non-empty
- Connection must be inbound and linked to this team

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Error Responses

Status	Meaning
400	Validation error, connection not inbound, or not linked
403	Insufficient permissions
409	Rewrite index already exists for this connection + remote_team_name pair on another team
500	Internal error

DELETE `/api/v1/teams/{team_id}/rewrite` **TEAM ADMIN+**

Clear a remote team name rewrite.

Query Parameters

<code>connection</code>	Required. Connection name to clear the rewrite for.
-------------------------	---

Success Response - 200

```
{
  "status": "ok",
```

```
"team_id": "abc123...",  
"connection": "relay-from-a"  
}
```

Connection Prompts

Note

These endpoints are called by interactive buttons in prompt messages. They are not intended to be called directly.

POST `/api/v1/prompt/accept` **TEAM ADMIN+**

Accept button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."} 
```

Links inbound connection to team, deletes the prompt, and updates the prompt post message to show acceptance. Returns a `PostActionIntegrationResponse` .

POST `/api/v1/prompt/block` **TEAM ADMIN+**

Block button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."} 
```

Sets prompt state to `blocked` and updates the prompt post message. Connection stays unlinked until `reset-prompt` is used.

POST `/api/v1/prompt/channel/accept` **TEAM ADMIN+**

Accept button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."} 
```

Links inbound connection to channel, deletes prompt, and updates the post.

POST `/api/v1/prompt/channel/block` **TEAM ADMIN+**

Block button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."} 
```

Sets channel prompt state to `blocked` and updates the post.

Connection Requests

Note

These endpoints are called by interactive buttons in connection request DMs sent to System Admins. They are not intended to be called directly. Requests are created when a Team Admin runs

`/crossguard init-team` (or the REST equivalent) while the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled).

POST `/api/v1/request/approve` **SYSTEM ADMIN**

Approve a pending connection link request. Links the connection to the team, deletes the request, updates all admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."} 
```

Behavior

- Executes the team init (same as a direct System Admin link)
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse` .

POST `/api/v1/request/deny` **SYSTEM ADMIN**

Deny a pending connection link request. Opens an interactive dialog for the admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."} 
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/request/deny-submit`

Returns a `PostActionIntegrationResponse` .

POST

`/api/v1/request/deny-submit`

SYSTEM ADMIN

Dialog submission handler for deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state : team_id|conn_key`
- `submission.reason` : optional denial reason text

Behavior

- Deletes the pending request
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Channel Connection Requests

When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can submit channel-level connection link requests. These endpoints handle the Team Admin approval workflow. They mirror the team-level request endpoints but operate on channel connections with Team Admins as the approvers.

POST**/api/v1/channel-request/approve****TEAM ADMIN+**

Approve a pending channel connection link request. Links the connection to the channel, deletes the request, updates all Team Admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"channel_id": "...", "team_id": "...", "conn_key": "..."} 
```

Behavior

- Validates the connection still exists in config and the team still has it linked
- Validates the target channel still exists
- Executes the channel init (same as a direct Channel Admin link)
- Removes Approve/Deny buttons from all Team Admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config or unlinked from the team while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse` .

POST**/api/v1/channel-request/deny****TEAM ADMIN+**

Deny a pending channel connection link request. Opens an interactive dialog for the Team Admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"channel_id": "...", "team_id": "...", "conn_key": "..."} 
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/channel-request/deny-submit`

Returns a `PostActionIntegrationResponse` .

POST

`/api/v1/channel-request/deny-submit`

TEAM ADMIN+

Dialog submission handler for channel request deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state` : `channel_id|team_id|conn_key`
- `submission.reason` : optional denial reason text

Behavior

- Deletes the pending channel connection request
- Removes Approve/Deny buttons from all Team Admin DM posts for this request
- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Dialog Handling

POST

`/api/v1/dialog/select-connection`

AUTHENTICATED

Interactive dialog handler when user selects a connection from the dropdown.

Request Body

`SubmitDialogRequest` with:

- `connection_name` field: selected connection in `direction:name` format
- `state` field: `action:target_id` (e.g., `init-team:abc123` , `teardown-channel:xyz789`)

Supported Actions

Action	Description
<code>init-team</code>	Link connection to team
<code>teardown-team</code>	Unlink connection from team
<code>init-channel</code>	Link connection to channel
<code>teardown-channel</code>	Unlink connection from channel

Validates permissions, performs the action, and sends an ephemeral message with the result. Returns `200` on success.

Message Transport Interface

Cross Guard supports NATS, Azure Queue Storage, Azure Blob Storage, and Azure Service Bus as transport layers for relaying messages between Mattermost instances. For the complete wire protocol reference, including all message envelope schemas, payload definitions, connection configuration, data flow diagrams, and full wire format examples, see the dedicated [Transport Interface Reference](#).

Error Codes

Reference for every numeric `error_code` emitted by Cross Guard logs

How to Read This Page

Every `p.API.Log*` call in the Cross Guard plugin (non-test code) carries a stable numeric `error_code` as its first key-value pair. Operators can grep production logs for the integer value even if the Go constant name changes later. Numbers are never reused or renumbered once assigned.

Example log line:

```
{"level":"error","msg":"Failed to check channel connections","error_code":10000,"chan
```

Codes are allocated per source file in non-overlapping 1000-wide ranges. The first two digits identify the subsystem. Each row below lists the exact log message, the fields you can expect alongside `error_code` , and the level at which the message is emitted (`DEBUG` , `INFO` , `WARN` , `ERROR`).

Informational and warning codes

Not every code represents a failure. Some mark successful events (connection established, subscription started, retry succeeded) or expected drops (policy filters, max retries reached). Check the log level first: `INFO` lines never require action, `WARN` lines are usually recoverable, and `ERROR` lines indicate a broken operation.

Range Quick Reference

Range	Source file	Subsystem	Count
10000–10999	hooks.go	Mattermost post and reaction event hooks	12

Range	Source file	Subsystem	Count
11000– 11999	api.go	REST API handlers	17
12000– 12999	configuration.go	Plugin configuration lifecycle	3
13000– 13999	command.go	Slash command handlers	1
14000– 14999	service.go	Team and channel initialization service	34
15000– 15999	inbound.go	Inbound message dispatch and file watcher	46
16000– 16999	connections.go	Outbound publisher and file upload	14
17000– 17999	sync_user.go	Synthetic relay-user resolution	4
18000– 18999	azure_blob_provider.go	Azure Blob WAL provider	50
19000– 19999	azure_provider.go	Azure Queue provider	11
20000– 20999	nats_provider.go	NATS transport provider	4
21000– 21999	prompt.go	Inbound connection prompts	17
22000– 22999	retry_dispatch.go	Missing-parent post retry queue	5
23000– 23999	store/caching.go	Cluster cache invalidation	1

Range	Source file	Subsystem	Count
24000– 24999	request.go	Team admin connection request workflow	17
25000– 25999	channel_request.go	Channel admin connection request workflow	18
26000– 26999	azure_servicebus_provider.go	Azure Service Bus provider	8

hooks.go (10000–10999)

Emitted from Mattermost post and reaction lifecycle hooks (`MessageHasBeenPosted` , `MessageHasBeenUpdated` , `MessageHasBeenDeleted` , `ReactionHasBeenAdded` , `ReactionHasBeenRemoved`). These run on every message event on the server, so transient failures here can prevent relay of individual messages.

Code	Name / Level	Log message and context	Troubleshooting
10000	<code>HooksChannelConnCheckFailed</code> ERROR	"Failed to check channel connections" with <code>channel_id</code> . The post hook tried to look up which Cross Guard connections are linked to the channel where a message was just posted, and the KV store lookup failed.	Check the <code>error</code> field for the underlying KV store error. Usually transient (store restart, read pressure). If repeated for the same <code>channel_id</code> run <code>/crossguard status</code> in that channel to compare what the store returns. If the store is corrupt, remove and re-add the channel link with <code>teardown-channel</code>

Code	Name / Level	Log message and context	Troubleshooting
			then <code>init-channel</code> .
10001	HooksGetChannelFailed ERROR	"Failed to get channel for relay" with <code>channel_id</code> . Mattermost returned an error when the hook asked for the channel record by ID.	Channel may have been deleted between the event and the hook firing which is safe to ignore. If persistent check Mattermost database health and that the plugin has read-channel permission. Grep for <code>channel_id</code> to see whether one specific channel is affected.
10002	HooksTeamConnCheckFailed ERROR	"Failed to check team connections" with <code>team_id</code> . The hook tried to read the team-level connection list before relaying a message.	Same pattern as 10000: inspect the adjacent <code>error</code> field. The team may have a partial or corrupt link. Running <code>/crossguard status</code> in the team confirms whether the link is readable.
10003	HooksGetTeamFailed ERROR	"Failed to get team for relay" with <code>team_id</code> . Mattermost returned an error when the hook asked for the team record.	Team may have been deleted, or Mattermost is under load. Check database health. Transient single occurrences are safe to ignore.
10004	HooksRelaySemaphoreFull WARN	"Relay semaphore full, dropping event" with <code>context</code> . The per-process	Raise <code>RelayConcurrent</code> in System Console

Code	Name / Level	Log message and context	Troubleshooting
		relay concurrency semaphore was saturated when a new event arrived, so the message was dropped rather than blocking the hook.	Plugins > Cross Guard. Look for correlated slowdowns in <code>connections</code>. logs (16001, 16006, 16007): the downstream provider is likely slow or unreachable. Sustained semaphore-full events mean the plugin cannot keep up with the local message rate.
10005	HooksGetUserForPostFailed ERROR	"Failed to get user for relay" with <code>user_id</code>. The create-post hook failed to load the author before serializing the envelope.	User may have been deleted, or the plugin lacks read-user permission. Check the <code>error</code> field. Occasional failures are safe to ignore because the next message from the same user will retry.
10006	HooksGetUserForUpdateFailed ERROR	"Failed to get user for relay" with <code>user_id</code>. Same as 10005, but from the edit-post hook.	Same as 10005.

Code	Name / Level	Log message and context	Troubleshooting
10007	HooksDeleteFlagCheckFailed ERROR	"Failed to check delete flag, skipping relay to avoid loop" with <code>post_id</code> . The delete hook could not check the <code>crossguard_delete_flag</code> used to break relay loops, so it skipped relaying this delete.	Inspect the KV store error. A persistent failure means delete events stop propagating. Check that the KV store is reachable and not out of quota. The flag has a short TTL so transient failures self-heal.
10008	HooksGetPostForReactAddFailed ERROR	"Failed to get post for reaction relay" with <code>post_id</code> . The reaction-add hook could not load the target post.	Post may have been deleted concurrently. Safe to ignore single instances. If repeated, check Mattermost post store health.
10009	HooksGetUserForReactAddFailed ERROR	"Failed to get user for reaction relay" with <code>user_id</code> . Reaction-add hook could not load the reacting user.	Same as 10005.
10010	HooksGetPostForReactRemFailed ERROR	"Failed to get post for reaction relay" with <code>post_id</code> . Reaction-remove hook could not load the target post.	Same as 10008.
10011	HooksGetUserForReactRemFailed ERROR	"Failed to get user for reaction relay" with <code>user_id</code> . Reaction-remove hook could not load the user whose reaction was removed.	Same as 10005.

Emitted from REST API handlers. Most codes correspond to a System Console action (test connection, set rewrite) or to a slash-command action dispatched via the API.

Code	Name / Level	Log message and context	Troubleshooting
11000	APINATSTestConnectFailed ERROR	"NATS connection test failed" with <code>address</code> . The <code>/test-connection</code> handler could not dial the NATS broker with the supplied configuration.	Verify network reachability settings, and authentication credentials (username/password) in the plugin host, try <code>telnet <host> <port></code> or <code>openssl s_client -connect <host>:<port></code> to test the connection.
11001	APIBuildTestMessageFailed ERROR	"Failed to build test message". The test-connection path could not construct the test envelope.	Internal bug. Capture the log message and file an issue. Unless the plugin is up-to-date, update it.
11002	APIPublishTestMessageFailed ERROR	"Failed to publish test message" with <code>subject</code> . Dialing succeeded but the publish call failed.	Verify that the NATS account has the correct permission on the connection. Check that the subject starts with a valid prefix. Check broker-side log operations.
11003	APIFlushTestMessageFailed ERROR	"Failed to flush test message" with <code>subject</code> . Publish was buffered but flush (forcing delivery) failed, usually because the connection dropped.	Retry the test. If reproducible, check the NATS account permissions. Check TLS hostname verification, keepalive settings, and firewalls.
11004	APITestMessageSent INFO	"Test message sent" with <code>subject</code> , <code>msg_id</code> .	Informational. The <code>msg_id</code> is correlated with the test connection.

Code	Name / Level	Log message and context	Troubleshooting
		Informational confirmation that the outbound test envelope was published.	<code>InboundTestMessage</code> (15011) on the receiver
11005	<code>APISubscribeInboundTestFailed</code> ERROR	"Failed to subscribe for inbound test" with <code>subject</code> . The inbound test-connection path could not subscribe to the NATS subject.	Verify the account has on the subject. Multi-t are often scoped to sp
11006	<code>APIFlushNATSConnFailed</code> ERROR	"Failed to flush NATS connection". Cleanup flush failed on the short-lived test client.	Non-fatal if the primar success. Indicates the early.
11007	<code>APIInboundTestSubscribeOK</code> INFO	"Inbound test subscription successful" with <code>subject</code> . Informational.	Informational.
11008	<code>APIAzureQueueTestFailed</code> ERROR	"Azure Queue connection test failed". The Azure Queue test could not reach or authenticate against the configured queue endpoint.	Check <code>queue_service_account_name</code> , <code>acc</code> <code>queue_name</code> . For Azi emulator is running an Azurite format (<code>http://host:10001</code>) For production, confir the account key has n
11009	<code>APIAzureBlobTestFailed</code> ERROR	"Azure Blob connection test failed". The Azure Blob test could not reach or authenticate	Check <code>service_url</code> <code>blob_container_name</code> container already exist credentials can create the blob emulator is ru

Code	Name / Level	Log message and context	Troubleshooting
		against the configured blob container.	(default 10000). See A the full Azure setup.
11010	APIGetUserFailed ERROR	"Failed to get user" with <code>user_id</code> . An API handler could not load the requesting user from Mattermost.	Usually means an expi transient store error. F the webapp. If repeate check Mattermost sto
11011	APIInitChannelGetTeamConns ERROR	"Failed to get team connections" with <code>team_id</code> . Channel-init endpoint could not load the owning team's connections (needed to auto-init the team).	Confirm the team exist <code>/crossguard init-t</code> Inspect the KV store e
11012	APITeardownChannelGetChanConns ERROR	"Failed to get channel connections" with <code>channel_id</code> . Channel-teardown endpoint could not load the channel's current connections.	Inspect the KV store e
11013	APITeardownTeamGetTeamConns ERROR	"Failed to get team connections" with <code>team_id</code> . Team-teardown endpoint could not load the team's connections.	Inspect the KV store e
11014	APIBulkChannelConnectionsFailed WARN	"Failed to get channel connections for bulk lookup" with <code>channel_id</code> . The bulk channel	Single failures only aff are safe to ignore. Rep many channels mean t The rest of the batch i

Code	Name / Level	Log message and context	Troubleshooting
		connection lookup used by the webapp channel indicator failed for one channel in the batch.	
11015	APITeamRewriteSet INFO	"Team rewrite set" with <code>team_id</code> , <code>connection</code> , <code>remote_team_name</code> , <code>user</code> . Audit log of a successful team name rewrite.	Informational and audi field is the admin who
11016	APITeamRewriteCleared INFO	"Team rewrite cleared" with <code>team_id</code> , <code>connection</code> , <code>user</code> . Audit log of a cleared rewrite.	Informational and audi

configuration.go (12000-12999)

Emitted from plugin configuration load and validation.

Code	Name / Level	Log message and context	Troubleshooting
12000	ConfigSameConfigPassed WARN	"setConfiguration called with the existing configuration". The <code>OnConfigurationChange</code> hook fired but Mattermost passed the same config object that is already loaded.	Usually benign, fired during plugin activation and config reloads. Spammy occurrences suggest a tight loop calling <code>SavePluginConfig</code> . Check for misbehaving admin integrations.

Code	Name / Level	Log message and context	Troubleshooting
12001	ConfigValidationWarn WARN	"Plugin configuration has validation warnings". Non-fatal validation warnings were raised while parsing plugin settings.	Read the warning text in the adjacent <code>warnings</code> field (connection with obsolete field, duplicate name, etc.). Edit the connection in System Console > Plugins > Cross Guard and save again to clear.
12010	AzureSPAuditConstructed INFO	"Azure auth (service-principal): credential constructed" with <code>provider</code> (one of <code>azure-queue</code> , <code>azure-blob</code> , <code>azure-servicebus</code>), <code>auth_mode</code> (always <code>service-principal</code>), <code>tenant_id</code> , <code>client_id</code> , <code>azure_cloud</code> , and <code>secret_hash_prefix</code> (first 12 hex chars of SHA-256 of the resolved secret). Emitted once per Service Principal credential construction so operators have a record of which AAD identity authenticated each connection. The secret value itself is never logged.	Audit-only; not an error. Use <code>secret_hash_prefix</code> to correlate which secret was active during an incident without exposing material that could be used to reconstruct it.

command.go (13000-13999)

Emitted from slash command handlers.

Code	Name / Level	Log message and context	Troubleshooting
13000	CommandOpenConnDialogFailed ERROR	"Failed to open connection dialog". The user ran a slash command (e.g., <code>/crossguard init-team</code>) without arguments and the plugin failed to open the interactive selection dialog.	Verify the Mattermost site URL is set correctly and that the server can reach its own plugin dialog endpoint. Check the <code>error</code> field for the specific RPC failure. Users can work around by passing the connection name directly on the command line.

service.go (14000-14999)

Emitted from the team and channel initialization service. This layer persists the team-to-connection and channel-to-connection links in the KV store, writes init and teardown announcement posts, and enforces header prefixes on linked channels.

Code	Name / Level	Log message and context	Troubleshooting
14000	ServiceInitTeamGetConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . <code>init-team</code> could not read the team's existing connection list before adding a new one.	Inspect the KV store. Usually a store outage with newly restored data without the plugin t init command is safe; idempotent; retry at is healthy.
14001	ServiceAddTeamConnFailed ERROR	"Failed to add team connection" with <code>team_id</code> , <code>conn</code> . <code>init-</code>	Inspect the KV store after the store recovery operation partially v

Code	Name / Level	Log message and context	Troubleshooting
		team failed to write the new team-to-connection link.	running init-team safe.
14002	ServiceInitTeamReReadConnsFailed ERROR	"Failed to re-read team connections" with team_id . After writing the link, the service could not re-read the team's connections to confirm the update.	The write may still have succeeded. Run /c status in the team /api/v1/teams/< to verify.
14003	ServiceAddTeamInitializedFailed ERROR	"Failed to add team to initialized list" with team_id . Failed to update the "initialized teams" index used by global status.	Inspect the KV store; primary link is still in global status may not be team. Re-running : fixes the index.
14004	ServicePostTeamInitMsgFailed WARN	"Failed to post initialization message". The service successfully linked the team but failed to post the confirmation message in town-square.	Non-fatal. Confirm that a town-square character (default), and that the Guard bot has posted Team initialization is complete.

Code	Name / Level	Log message and context	Troubleshooting
14005	ServiceCheckTeamStatusFailed ERROR	"Failed to check team status" with <code>team_id</code> . Internal status lookup failed while the service was answering a status query.	Inspect the KV store. Affects <code>/crossguard</code> output.
14006	ServiceGetInitializedTeamsFailed ERROR	"Failed to get initialized teams". Global status failed to load the list of initialized teams.	Inspect the KV store. Affects <code>GET /api/</code>
14007	ServiceTeamStatusLookupTeamFailed WARN	"Failed to look up team for status" with <code>team_id</code> . Status endpoint failed to load a team record.	Team may have been removed since initialization. If <code>teardown-team</code> to manually remove the team.
14008	ServiceTeamStatusGetConnsFailed WARN	"Failed to get team connections for status" with <code>team_id</code> . Status endpoint failed to load the team's connections.	Inspect the KV store.
14009	ServiceParseOutboundConnFailed WARN	"Failed to parse outbound connection configuration". A stored team link references an outbound connection that	The connection was deleted in System Console being linked. The link is as <code>orphaned: true</code> output. Run <code>teardown</code> with the old name, or the connection in System Console.

Code	Name / Level	Log message and context	Troubleshooting
		could not be parsed from the current plugin config.	
14010	ServiceParseInboundConnFailed WARN	"Failed to parse inbound connection configuration". Same as 14009, but for inbound connections.	Same as 14009.
14011	ServiceTeardownGetChanConnsFailed ERROR	"Failed to get channel connections" with <code>channel_id</code> . During team teardown, failed to list channel connections nested under the team.	Inspect the KV store. Orphaned channel IDs remain and can be cleaned up with explicit <code>teardown-channel</code> calls.
14012	ServiceTeardownGetTeamConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . Team teardown failed to load the team's own connection list.	Inspect the KV store.
14013	ServiceInitChanGetTeamConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . <code>init-channel</code> could not verify	Confirm the team is initialized (<code>/crossguard status</code>). If not, run <code>init-team</code> .

Code	Name / Level	Log message and context	Troubleshooting
		the parent team's connections.	
14014	ServiceInitChanGetChanConnsFailed ERROR	"Failed to get channel connections" with <code>channel_id</code> . <code>init-channel</code> could not read the channel's existing connections.	Inspect the KV store
14015	ServiceAddChanConnFailed ERROR	"Failed to add channel connection" with <code>channel_id</code> , <code>conn</code> . Failed to write a new channel-to-connection link.	Inspect the KV store command is idempotent
14016	ServiceInitChanReReadConnsFailed ERROR	"Failed to re-read channel connections" with <code>channel_id</code> . After linking, re-read failed.	Verify with <code>/cross status</code> in the character
14017	ServiceChanHeaderPrefixFailed WARN	"Failed to update channel header with CrossGuard prefix" with <code>channel_id</code> . The channel was linked but the header could not be updated to add the Cross Guard marker.	Verify the bot has n channel permission target channel. The still active; only the marker is missing.

Code	Name / Level	Log message and context	Troubleshooting
14018	ServicePostChanInitMsgFailed WARN	"Failed to post channel init message". Channel was linked but the confirmation message could not be posted.	Confirm the bot car target channel. Non
14019	ServiceRemoveChanGetConnsFailed ERROR	"Failed to get channel connections" with <code>channel_id</code> . <code>teardown-channel</code> could not load the current connections.	Inspect the KV stor
14020	ServiceRemoveChanConnFailed ERROR	"Failed to remove channel connection" with <code>channel_id</code> , <code>conn</code> . Failed to delete a single channel-to-connection link.	Inspect the KV stor retry. Orphaned link removed by re-run <code>teardown-channel</code>
14021	ServiceRemoveChanReReadConnsFailed ERROR	"Failed to re-read channel connections" with <code>channel_id</code> . Post-removal verify read failed.	Verify with <code>/cross status</code> .
14022	ServiceDeleteChanConnsFailed ERROR	"Failed to delete channel connections" with <code>channel_id</code> .	Inspect the KV stor retry teardown.

Code	Name / Level	Log message and context	Troubleshooting
		Failed to delete the channel's full connection record during teardown.	
14023	ServiceChanHeaderRemovePrefixFailed WARN	"Failed to remove CrossGuard prefix from channel header" with <code>channel_id</code> . Could not strip the Cross Guard marker from the channel header during teardown.	Verify the bot has n channel permission can edit the header remove the leftover
14024	ServicePostChanTeardownMsgFailed WARN	"Failed to post channel teardown message". Could not post the "channel unlinked" confirmation message.	Non-fatal. Confirm post.
14025	ServiceTeardownTeamGetConnsFailed ERROR	"Failed to get team connections" with <code>team_id</code> . Team teardown failed to load connections during cleanup.	Inspect the KV stor

Code	Name / Level	Log message and context	Troubleshooting
14026	ServiceRemoveTeamConnFailed ERROR	"Failed to remove team connection" with <code>team_id</code> , <code>conn</code> . Failed to delete a team-to-connection link during teardown.	Inspect the KV store and retry.
14027	ServiceTeardownTeamReReadConnsFailed ERROR	"Failed to re-read team connections" with <code>team_id</code> . Post-removal verify read failed.	Verify with <code>/cross status</code> .
14028	ServiceRemoveTeamInitializedFailed ERROR	"Failed to remove team from initialized list" with <code>team_id</code> . Failed to update the "initialized teams" index during teardown.	Inspect the KV store. Global status may sync team until re-run of
14029	ServicePostTeamTeardownMsgFailed WARN	"Failed to post team teardown message". Non-fatal.	Confirm bot can post square.
14030	ServiceGlobalParseOutConnFailed WARN	"Failed to parse outbound connections". Global status could not parse an outbound connection from plugin config when answering	Review the outbound connection form in Console. The adjacent field names the specific failure.

Code	Name / Level	Log message and context	Troubleshooting
		GET /api/v1/status .	
14031	ServiceGlobalParseInConnFailed WARN	"Failed to parse inbound connections". Same as 14030 for inbound connections.	Same as 14030.
14032	ServiceMapParseOutConnFailed WARN	"Failed to parse outbound connections for map". Startup failed to build the internal outbound connection map.	Inspect the preceding error. The failing code can be skipped from the log until fixed.
14033	ServiceMapParseInConnFailed WARN	"Failed to parse inbound connections for map". Same as 14032 for inbound connections.	Same as 14032.

inbound.go (15000-15999)

Emitted from the inbound dispatcher that consumes messages from the transport, applies idempotency, and materializes posts, edits, deletes, and reactions locally. Also covers the inbound file watcher loop.

Code	Name / Level	Log message and context	Troubleshooting
15000	InboundParseConnsFailed ERROR	"Failed to parse inbound connections". Startup failed to parse any inbound	Review all connections in the Console. Flagged in

Code	Name / Level	Log message and context	Troubleshooting
		connections from plugin config.	<code>error</code> file at least one plugin inbound connection messages received.
15001	InboundConnectFailed ERROR	"Failed to connect inbound" with <code>name</code> , <code>provider</code> . An inbound connection failed to dial its transport (NATS, Azure Queue, or Azure Blob).	Check provider codes in the log context: 19xxx / 20xxx network credentials subject/queue names. The loop will reconfigured
15002	InboundSubscribeFailed ERROR	"Failed to subscribe inbound" with <code>name</code> . Connected to NATS but subscribe call failed.	Verify the plugin has subscription permission subject. Check side authentication
15003	InboundSubscriptionEstablished INFO	"Inbound subscription established" with <code>name</code> , <code>provider</code> . Informational confirmation of a successful inbound subscription.	Informational a liveness : plugin restart
15004	InboundRelaySemaphoreFull WARN	"Relay semaphore full, dropping inbound message" with <code>conn</code> . The inbound concurrency semaphore was saturated; the current message was dropped.	Raise <code>RelayCon</code> investigate slowness: <code>CreatePo</code> database locks hooks on the Sustained the receive up.

Code	Name / Level	Log message and context	Troubleshooting
15005	InboundUnmarshalFailed ERROR	"Failed to unmarshal inbound message" with <code>conn</code> . Received an envelope that parsed neither as JSON nor XML.	Check that uses a conversion and <code>message_</code> matches or Capture the envelope for
15006	InboundPostMissingPayload ERROR	"Inbound post: missing payload" with <code>conn</code> . A <code>post</code> envelope arrived with no payload section.	Sender bug or corruption. Capture the message to confirm sequence and file an
15007	InboundUpdateMissingPayload ERROR	"Inbound update: missing payload" with <code>conn</code> . A <code>post_update</code> envelope arrived with no payload.	Same as 15006
15008	InboundDeleteMissingPayload ERROR	"Inbound delete: missing payload" with <code>conn</code> . A <code>post_delete</code> envelope arrived with no payload.	Same as 15006
15009	InboundReactionAddMissingPayload ERROR	"Inbound reaction add: missing payload" with <code>conn</code> . A <code>reaction_add</code> envelope arrived with no payload.	Same as 15006
15010	InboundReactionRemoveMissingPayload ERROR	"Inbound reaction remove: missing payload" with <code>conn</code> . A <code>reaction_remove</code> envelope arrived with no payload.	Same as 15006
15011	InboundTestMessageReceivedWithID INFO	"Received inbound test message" with <code>conn</code> , <code>id</code> . The inbound test subscriber	Informational. Correlates

Code	Name / Level	Log message and context	Troubleshooting
		received a test envelope with an ID.	APITestM (11004) on
15012	InboundTestMessageReceived INFO	"Received inbound test message" with <code>conn</code> . Test envelope without an ID.	Information
15013	InboundUnknownMessageType WARN	"Unknown inbound message type" with <code>conn</code> , <code>type</code> . An envelope with an unrecognized <code>type</code> field arrived.	Usually me sender is t older recei the receive Check the plugin vers
15014	InboundMissingMessageQueueFull ERROR	"Missing message: queue full, dropping message" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>queue_size</code> . The retry queue for messages whose parent post was missing locally was full; the new message was dropped.	Increase th parent que settings, o why parent arriving. Se retry_dispa how the qu processes persistently usually me sender nev the parent
15015	InboundMissingMessageQueued WARN	"Missing message: queuing for retry" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>queue_size</code> . A message arrived whose parent post has not yet been received; it was queued for retry.	Normal dur traffic or o delivery. TI dispatcher (see 22000 Rising <code>que</code> across mar indicates o problems i transport.
15016	InboundPostResolveFailed WARN	"Inbound post: resolve failed" with <code>conn</code> . Failed to resolve the target	Most comr the remote does not r

Code	Name / Level	Log message and context	Troubleshooting
		team/channel for an inbound post.	team and r configured /crossgu rewrite-t Slash Com ensure tea match on k
15017	InboundPostIdempotencyLookupFailed WARN	"Inbound post: idempotency lookup failed, processing anyway" with <code>conn</code> , <code>postID</code> . The idempotency cache lookup failed, so the plugin cannot guarantee this post was not already created; it proceeds anyway.	Inspect the error. Risk: duplicate p message is by the tran
15018	InboundPostResolveUserFailed ERROR	"Inbound post: resolve user failed" with <code>conn</code> , <code>username</code> . Failed to resolve the post author to a local or sync user.	Check <code>Username</code> setting and sync-user succeeded username. <code>17xxx</code> cc the underly The post is
15019	InboundPostRootMappingLookupFailed WARN	"Inbound post: failed to look up root mapping" with <code>conn</code> , <code>remote_root_id</code> . A thread reply arrived but the root-post mapping lookup failed.	Inspect the error.
15020	InboundPostRootNotFoundStandalone WARN	"Inbound post: root not found after retries, creating standalone" with <code>conn</code> , <code>remote_root_id</code> . After the retry dispatcher gave up waiting for the parent, the	Indicates t never arriv may have f missed, or it). Check t logs for the

Code	Name / Level	Log message and context	Troubleshooting
		plugin created the reply as a standalone post at the top level of the channel.	<code>remote_reply_id</code> Consider retrying the message so slow arrivals are handled.
15021	InboundPostCreateFailed ERROR	"Inbound post: create failed" with <code>conn</code> . Mattermost rejected the <code>CreatePost</code> call.	Common causes: channel length over limit, characters archived or bot lacks permissions. Verify permission validation in the <code>error</code> .
15022	InboundPostStoreMappingFailed ERROR	"Inbound post: failed to store post mapping" with <code>conn</code> , <code>remote_id</code> , <code>local_id</code> . The post was created locally, but saving the remote-to-local mapping failed.	Inspect the error. Future deletes of posts may not find the mapping. Manual cleanup needed; check logs if this recurs.
15023	InboundUpdateMappingLookupFailed ERROR	"Inbound update: failed to look up post mapping" with <code>conn</code> , <code>remote_id</code> . Could not find the local post ID for an incoming edit.	Possible causes: original post not reached the server (check 15022), the mapping was dropped (15027-15028).
15024	InboundUpdateGetLocalPostFailed ERROR	"Inbound update: failed to get local post" with <code>conn</code> , <code>local_id</code> . Mapping existed but Mattermost could not load the local post.	Post may have been deleted locally. Ignore since the mapping exists.
15025	InboundUpdatePostFailed ERROR	"Inbound update: failed to update post" with <code>conn</code> , <code>local_id</code> . Mattermost	Common causes: deleted content, validation errors.

Code	Name / Level	Log message and context	Troubleshooting
		rejected the <code>UpdatePost</code> call.	lacks edit p Inspect the field.
15026	<code>InboundDeleteMappingLookupFailed</code> ERROR	"Inbound delete: failed to look up post mapping" with <code>conn</code> , <code>remote_id</code> . Could not find the local post ID for an incoming delete.	Same as 15
15027	<code>InboundDeleteSetFlagFailed</code> ERROR	"Inbound delete: failed to set delete flag" with <code>conn</code> , <code>local_id</code> . Failed to set the <code>crossguard_delete_flag</code> that prevents the delete from echoing back through the outbound hook.	Inspect the error. With the delete back to the creating a for 10007 c side.
15028	<code>InboundDeletePostFailed</code> ERROR	"Inbound delete: failed to delete post" with <code>conn</code> , <code>local_id</code> . Mattermost rejected the <code>DeletePost</code> call.	Post may a deleted. In <code>error</code> fie
15029	<code>InboundDeleteRemoveFlagFailed</code> WARN	"Inbound delete: failed to remove delete flag" with <code>conn</code> , <code>local_id</code> . Failed to clear the delete flag after a successful delete.	Non-fatal. TTL and ex automatica
15030	<code>InboundDeleteRemoveMappingFailed</code> WARN	"Inbound delete: failed to remove post mapping" with <code>conn</code> , <code>remote_id</code> . Leaves a stale mapping entry.	Inspect the error. Stale only a spac does not a correctnes
15031	<code>InboundReactionMappingLookupFailed</code> ERROR	"Inbound reaction: failed to look up post mapping" with <code>conn</code> , <code>remote_id</code> . Could	Same as 15

Code	Name / Level	Log message and context	Troubleshooting
		not find the local post for a reaction event.	
15032	InboundReactionResolveFailed WARN	"Inbound reaction: resolve failed" with <code>conn</code> . Failed to resolve the target channel for a reaction.	Same as 15031
15033	InboundReactionResolveUserFailed ERROR	"Inbound reaction: resolve user failed" with <code>conn</code> , <code>username</code> . Failed to resolve the reacting user.	Same as 15031
15034	InboundReactionAddFailed ERROR	"Inbound reaction: add failed" with <code>conn</code> , <code>post_id</code> . Mattermost rejected the <code>AddReaction</code> call.	Post may not exist, or the user may not have permission. Check the <code>error</code> field.
15035	InboundReactionRemoveFailed ERROR	"Inbound reaction: remove failed" with <code>conn</code> , <code>post_id</code> . Mattermost rejected the <code>RemoveReaction</code> call.	Same as 15031
15036	InboundFileWatcherStarted INFO	"File watcher started" with <code>conn</code> . The inbound file watcher goroutine started.	Informational message at plugin start. Each inbound connector will transfer events.
15037	InboundFileWatcherExited ERROR	"File watcher exited with error" with <code>conn</code> . The watcher loop exited, usually due to a fatal provider error.	Check the <code>error</code> field. It is correlated with 18xxx / 18xxx codes. The connector will restart on the next poll tick. Plugins should indicate connectivity or network outages.

Code	Name / Level	Log message and context	Troubleshooting
15038	InboundFileMissingHeaders WARN	"Inbound file: missing required headers, skipping" with <code>key</code> . A staged file blob lacked the expected Cross Guard metadata headers.	Something into the Container v envelope h Usually sta data. Capti by <code>key</code> fo
15039	InboundFileConnInactive WARN	"Inbound file: connection no longer active, skipping" with <code>conn</code> , <code>filename</code> . A file arrived for a connection that is no longer active on this instance.	Expected i after a <code>te</code> Persistent the sender relaying to connection
15040	InboundFileFilteredByPolicy INFO	"Inbound file filtered by policy" with <code>filename</code> , <code>conn</code> . A file was rejected by the connection's allow/deny extension filter.	Expected v are configu <code>file_fil</code> <code>file_fil</code> the connec filter was v
15041	InboundFileMappingLookupFailed ERROR	"Inbound file: post mapping lookup failed after retries" with <code>conn</code> , <code>remote_post_id</code> . Tried to find the local post ID for the file's parent post but the lookup kept failing.	Inspect the error. The t attached, a left in place (Azure) or (NATS).
15042	InboundFileNoMappingFound WARN	"Inbound file: no post mapping found after retries" with <code>conn</code> , <code>remote_post_id</code> . The mapping lookup succeeded (no store error) but no local post exists for the remote ID.	The parent arrived or v out. Check for the <code>remote_p</code> Usually tra out-of-ord

Code	Name / Level	Log message and context	Troubleshooting
15043	InboundFileGetLocalPostFailed ERROR	"Inbound file: failed to get local post" with <code>local_post_id</code> . Mapping existed but Mattermost could not load the local post.	Post may have been deleted. See 14030 .
15044	InboundFileUploadFailed ERROR	"Failed to upload file to Mattermost" with <code>filename</code> . Mattermost rejected the <code>UploadFile</code> call.	Check Mattermost <code>MaxFileSize</code> storage quota available during the <code>error</code> .
15045	InboundFileAttachFailed ERROR	"Failed to attach file to post" with <code>local_post_id</code> , <code>file_id</code> . File was uploaded but the association with the post failed.	Usually a problem with an invalid <code>post_id</code> . Inspect the <code>error</code> field.

connections.go (16000-16999)

Emitted from the outbound publisher pipeline: envelope serialization, multi-part splitting, provider publish with retries, and file upload to object store.

Code	Name / Level	Log message and context	Troubleshooting
16000	ConnectionsParseOutboundFailed ERROR	"Failed to parse outbound connections for relay". Failed to parse outbound connections at publish time.	Inspect the adjacent <code>error</code> field and fix the connection form in System Console. See also 14030 .

Code	Name / Level	Log message and context	Troubleshooting
16001	ConnectionsConnectOutboundFailed ERROR	"Failed to connect outbound for relay" with <code>name</code> , <code>provider</code> . Outbound provider dial failed when a message needed to be published.	Verify provider reachability and credentials. Check correlated <code>18xxx</code> / <code>19xxx</code> / <code>20xxx</code> codes. Publish will retry per <code>PublishRetries</code> before giving up (16007).
16002	ConnectionsOutboundEstablished INFO	"Outbound connection established for relay" with <code>name</code> , <code>provider</code> . Informational confirmation.	Informational. Useful as a liveness signal.
16003	ConnectionsSerializeFailed ERROR	"Failed to serialize outbound message" with <code>name</code> , <code>format</code> . Could not serialize the envelope to JSON or XML.	Internal bug. Capture the adjacent <code>error</code> field and the source post ID for diagnosis.
16004	ConnectionsMessageSplit INFO	"Message split into parts for provider size limit" with <code>connection</code> , <code>parts</code> , <code>originalSize</code> .	Informational. Frequent splits with high <code>parts</code> counts indicate very large posts; consider whether file transfer would be more efficient

Code	Name / Level	Log message and context	Troubleshooting
		A large message was split into multiple transport parts.	
16005	ConnectionsSerializePartFailed ERROR	"Failed to serialize message part" with <code>name</code> , <code>part</code> . Part of a split message failed to serialize.	Same as 16003.
16006	ConnectionsPublishPartFailed ERROR	"Failed to publish message part" with <code>name</code> , <code>part</code> . Publish of a single part failed on one attempt.	Transient if followed by successful retry. If all parts for a message fail see 16007.
16007	ConnectionsPublishAfterRetriesFail ERROR	"Failed to publish to outbound after retries" with <code>name</code> . All publish retries for a part were exhausted. The message was dropped.	Escalate: transport is persistently unavailable or rejecting messages. Check provider logs and credentials. Raising <code>PublishRetries</code> only helps if the failure is genuinely transient.

Code	Name / Level	Log message and context	Troubleshooting
16008	ConnectionsGetFileInfoFailed ERROR	"Failed to get file info for relay" with <code>file_id</code> , <code>post_id</code> . FileInfo lookup failed before upload.	File may have been removed. Inspect the <code>error</code> field.
16009	ConnectionsSkipOversizedFile WARN	"Skipping oversized file for relay" with <code>file</code> , <code>size</code> , <code>max</code> . A file exceeded the per-connection max size and was skipped.	Expected when limits are enforced. Raise the connection's max-file-size if intentional. The post is still relayed without the attachment
16010	ConnectionsDownloadFileFailed ERROR	"Failed to download file for relay" with <code>file_id</code> , <code>file</code> . Failed to download the local file bytes from Mattermost before upload.	Check Mattermost file storage health (S3, local disk, etc).
16011	ConnectionsOutboundFileFiltered INFO	"Outbound file filtered by policy" with <code>file</code> , <code>conn</code> . File was filtered out by the outbound extension filter.	Expected when filters are configured. Adjust the outbound filter if unintentional.
16012	ConnectionsFileSemaphoreFull WARN	"File semaphore full, skipping file	Raise the file transfer concurrency setting, or

Code	Name / Level	Log message and context	Troubleshooting
		upload" with <code>file</code> , <code>conn</code> . The file transfer concurrency semaphore was full.	investigate object store slowness. The post is still relayed without this attachment.
16013	ConnectionsUploadFileFailed ERROR	"Failed to upload file" with <code>key</code> , <code>conn</code> . Upload to JetStream Object Store or Azure Blob failed.	Check provider logs and credentials. The referenced file will not arrive on the receiving side; the post envelope still goes through.

sync_user.go (17000-17999)

Emitted from the synthetic relay user subsystem, which creates local Mattermost accounts to represent remote authors when `UsernameLookup` does not find a match.

Code	Name / Level	Log message and context	Troubleshooting
17000	SyncUserTruncatedUsername WARN	"Truncated sync username to fit limit" with <code>original</code> , <code>munged</code> . A remote username exceeded the Mattermost 64-char username limit and was shortened.	Normal for very long remote usernames. The sync user is still created under the truncated name. Users with the same truncated prefix will collide; consider naming conventions if this happens often.
17001	SyncUserLookupFallback DEBUG	"Username lookup did not find local user, falling back to sync	Expected when the remote username is not a local user. A sync

Code	Name / Level	Log message and context	Troubleshooting
		user" with <code>username</code> , <code>conn</code> . The <code>UsernameLookup</code> setting is enabled but no matching local user was found.	user is created and used instead.
17002	<code>SyncUserAddToTeamFailed</code> WARN	"Failed to add sync user to team" with <code>user_id</code> , <code>team_id</code> . Failed to add the new sync user to the target team.	Check team membership limits and that the team allows the sync user's email domain. The post will still attempt to proceed but may not appear until the user is a team member.
17003	<code>SyncUserAddToChannelFailed</code> WARN	"Failed to add sync user to channel" with <code>user_id</code> , <code>channel_id</code> . Failed to add the sync user to the target channel.	Check channel permissions and whether the channel is private or restricted. The post may not be created until the user is a channel member.

azure_blob_provider.go (18000-18999)

Emitted from the Azure Blob WAL provider. This provider batches outgoing envelopes into a write-ahead log on local disk, uploads them as blobs, stages deferred file attachments via a companion file, and uses lease-based locking so only one receiver per container processes a given segment. Most codes relate to WAL rotation, recovery, or lock contention.

Code	Name / Level	Log message and context
18000	<code>AzureBlobContainerCreateRetry</code> WARN	"Azure Blob: transient container create c retrying" with <code>container</code> , <code>attempt</code> ,

Code	Name / Level	Log message and context
		<code>delay</code> . Container creation was retried startup.
18001	AzureBlobWALInTempStorage WARN	"Azure Blob provider: WAL directory is in temp storage and may not survive container restarts" with <code>wal_dir</code> . The WAL directory resolved to <code>\$TMPDIR</code> because no persistent path is configured.
18002	AzureBlobWALDirFsyncFailed WARN	"Azure Blob: WAL dir fsync failed" with <code>path</code> . Fsync of the WAL directory failed.
18003	AzureBlobMarshalPendingFailed WARN	"Azure Blob: failed to marshal pending files. Could not serialize pending file refs when preparing a WAL flush.
18004	AzureBlobWriteCompanionFailed ERROR	"Azure Blob: failed to write companion files.json" with <code>path</code> . The companion file tracks deferred file uploads for a WAL segment; writing it failed.
18005	AzureBlobShutdownLeftWALFiles WARN	"Azure Blob: shutdown left WAL files for recovery" with <code>count</code> , <code>wal_dir</code> . At shutdown, this many WAL files had not yet been uploaded.

Code	Name / Level	Log message and context
18006	AzureBlobWALFsyncRotationFailed WARN	"Azure Blob: WAL fsync failed during rotation" with <code>path</code> . Fsync during WAL segment rotation failed.
18007	AzureBlobWALCloseRotationFailed ERROR	"Azure Blob: WAL close failed during rotation skipping upload" with <code>path</code> . The close rotate failed; the segment is skipped for rotation cycle.
18008	AzureBlobWALUploadFailed ERROR	"Azure Blob: WAL upload failed, leaving recovery" with <code>path</code> . Uploading a rotational WAL segment to Azure Blob failed.
18009	AzureBlobDeleteWALFailed WARN	"Azure Blob: failed to delete WAL after upload" with <code>path</code> . Segment uploaded successfully but local delete failed.
18010	AzureBlobMarshalFailedRefsFailed ERROR	"Azure Blob: failed to marshal failed refs companion rewrite" with <code>path</code> , <code>count</code> . Internal marshal error during companion rewrite.
18011	AzureBlobRewriteCompanionFailed ERROR	"Azure Blob: failed to rewrite companion files.json with failed refs" with <code>path</code> , <code>count</code> . Could not rewrite the companion with the remaining failed refs after a partial upload.
18012	AzureBlobDeleteCompanionFailed WARN	"Azure Blob: failed to delete companion files.json" with <code>path</code> . All deferred uploads completed but deleting the companion failed.
18013	AzureBlobDeferredFileFetchFailed ERROR	"Azure Blob: deferred file fetch failed" with <code>file_id</code> , <code>post_id</code> . Failed to re-read local Mattermost file for deferred upload.

Code	Name / Level	Log message and context
18014	AzureBlobDeferredFileUploadFailed ERROR	"Azure Blob: deferred file upload failed" with <code>file_id</code> , <code>post_id</code> . Failed to upload deferred file attachment to the Azure Blob container.
18015	AzureBlobShutdownMarshalResidualFailed ERROR	"Azure Blob: shutdown: failed to marshal residual pending files" with <code>count</code> . At shutdown, failed to serialize residual per state.
18016	AzureBlobShutdownPersistResidualFailed ERROR	"Azure Blob: shutdown: failed to persist residual pending files" with <code>count</code> , <code>path</code> . At shutdown, failed to write the residual file to disk.
18017	AzureBlobShutdownPersistedResidual WARN	"Azure Blob: shutdown persisted residual pending files for recovery" with <code>count</code> , <code>path</code> . Residual pending state was successfully persisted for next startup.
18018	AzureBlobListFailed ERROR	"Azure Blob: list failed" with <code>container</code> . The receiver could not list blobs in the V container.
18019	AzureBlobDeleteRetryFailed WARN	"Azure Blob: delete retry failed (marker present)" with <code>blob</code> . Retry of a blob delete (after processed-marker write) failed.
18020	AzureBlobDownloadFailed WARN	"Azure Blob: download failed" with <code>blob</code> . Failed to download a WAL segment blob during receiver poll.
18021	AzureBlobHandlerError WARN	"Azure Blob: handler error, will retry blob" with <code>blob</code> . The downstream inbound handler returned an error for this segment.

Code	Name / Level	Log message and context
18022	AzureBlobDeleteFailed WARN	"Azure Blob: delete failed" with blob . I to delete a WAL segment after successful processing.
18023	AzureBlobWriteProcessedMarkerFailed WARN	"Azure Blob: failed to write processed marker" with blob . Could not write the "processed" marker that prevents reprocessing.
18024	AzureBlobClearProcessedMarkerFailed WARN	"Azure Blob: failed to clear processed marker" with blob . Cleanup of a stale processed marker failed.
18025	AzureBlobLockGetFailed WARN	"Azure Blob: lock get failed" with blob Failed to read the receiver lock blob.
18026	AzureBlobLockTokenGenFailed WARN	"Azure Blob: lock token generation failed with blob . Failed to generate a random token (crypto rand failure).
18027	AzureBlobLockSetFailed WARN	"Azure Blob: lock set failed" with blob Failed to write the receiver lock blob.
18028	AzureBlobCorruptLockReclaimFailed WARN	"Azure Blob: corrupt lock reclaim failed" blob . Tried to reclaim a corrupt (unparseable) lock blob.

Code	Name / Level	Log message and context
18029	AzureBlobStaleLockReclaimFailed WARN	"Azure Blob: stale lock reclaim failed" with <code>blob</code> . Tried to reclaim a stale lock (owner died without releasing) and failed to overwrite.
18030	AzureBlobLockReleaseFailed WARN	"Azure Blob: lock release failed" with <code>blob</code> . Failed to release the receiver lock during shutdown.
18031	AzureBlobLockReleaseGetFailed WARN	"Azure Blob: lock release get failed" with <code>blob</code> . Failed to read the lock blob during release to verify ownership.
18032	AzureBlobLockReleaseCorrupt WARN	"Azure Blob: lock release saw corrupt value leaving" with <code>blob</code> . At release, the lock contents were corrupt and were left for reclaim.
18033	AzureBlobLockReleaseReclaimedByOther WARN	"Azure Blob: skipping release, lock reclaimed by another node" with <code>blob</code> , <code>current_node</code> . Another instance reclaimed the lock before this one released it.
18034	AzureBlobLockReleaseFailed2 WARN	"Azure Blob: lock release failed" with <code>blob</code> . Second-stage lock release (delete of the blob) failed.
18035	AzureBlobWALRecoveryScanRootFailed WARN	"Azure Blob: WAL recovery: failed to scan root" with <code>root</code> . Startup recovery failed to scan the root of the WAL directory.
18036	AzureBlobWALRecoveryScanDirFailed WARN	"Azure Blob: WAL recovery: failed to scan directory" with <code>dir</code> . Failed to scan a WAL subdirectory during recovery.
18037	AzureBlobWALRecoverySkipUnrecognized WARN	"Azure Blob: WAL recovery: skipping unrecognized file" with <code>path</code> . Recovering

Code	Name / Level	Log message and context
		skipped a file with an unknown name pa
18038	AzureBlobWALRecoveryUploadFailed ERROR	"Azure Blob: WAL recovery upload failed with <code>path</code> . Recovery failed to upload a leftover WAL segment.
18039	AzureBlobWALRecoveryDeleteFailed WARN	"Azure Blob: WAL recovery delete failed <code>path</code> . After recovery upload, the local segment delete failed.
18040	AzureBlobWALRecoveryReadCompanionFail WARN	"Azure Blob: WAL recovery: failed to read companion file" with <code>path</code> . Recovery failed to read a segment's companion file.
18041	AzureBlobWALRecoveryMalformedCompanion WARN	"Azure Blob: WAL recovery: malformed companion file" with <code>path</code> . A companion file exists but is unparseable.
18042	AzureBlobWALRecoveryMarshalRemaining WARN	"Azure Blob: WAL recovery: failed to marshal remaining refs" with <code>path</code> , <code>count</code> . Internal error during recovery companion rewrite
18043	AzureBlobWALRecoveryRewriteCompanion WARN	"Azure Blob: WAL recovery: failed to rewrite companion file" with <code>path</code> , <code>count</code> . Could not rewrite a recovered companion file.
18044	AzureBlobFileListFailed ERROR	"Azure Blob: file list failed" with <code>container</code> . Failed to list deferred file blobs during receiver polling.
18045	AzureBlobFileDeleteRetryFailed WARN	"Azure Blob: file delete retry failed (mark present)" with <code>blob</code> . Retry of deferred blob delete failed.
18046	AzureBlobFileDownloadFailed WARN	"Azure Blob: file download failed" with <code>blob</code> . Failed to download a deferred file blob.

Code	Name / Level	Log message and context
18047	AzureBlobFileHandlerError WARN	"Azure Blob: file handler error" with blob The file handler rejected a downloaded deferred file.
18048	AzureBlobFileDeleteFailed WARN	"Azure Blob: file delete failed" with blob Failed to delete a deferred file blob after successful processing.
18049	AzureBlobSPCredentialFailed ERROR	"Azure Blob: service principal credential failed" with tenant_id , client_id , sanitized error . azidentity.NewClientSecretCredential rejected the supplied Service Principal credentials at provider construction time

azure_provider.go (19000-19999)

Emitted from the Azure Queue provider, which uses an Azure Storage Queue for envelopes and a paired Azure Blob container for file attachments.

Code	Name / Level	Log message and context	T
19000	AzureQueueCreateQueueFailed WARN	"Azure Queue: could not create queue (may already exist)" with queue . Idempotent startup create returned an error.	U e c c p
19001	AzureQueueCreateContainerFail WARN	"Azure Blob: could not create container (may already exist)" with container . Idempotent blob container create returned an error.	S

Code	Name / Level	Log message and context	7
19002	AzureQueueDequeueFailed ERROR	"Azure Queue dequeue failed" with <code>queue</code> . Failed to dequeue a batch of messages.	C c s c r ir
19003	AzureQueueDecodeFailed ERROR	"Azure Queue: failed to decode message" with <code>queue</code> . A dequeued message body was not a valid Cross Guard envelope.	C s c v
19004	AzureQueueHandlerRetry WARN	"Azure Queue: handler returned error, message will retry" with <code>queue</code> . The downstream inbound handler returned an error; the message is left on the queue for the next visibility timeout.	C S n c n
19005	AzureQueueDeleteProcessedFail WARN	"Azure Queue: failed to delete processed message" with <code>queue</code> . Message was processed but the delete call failed.	M a h t c
19006	AzureQueueBlobListFailed ERROR	"Azure Blob list failed" with <code>container</code> . Failed to list deferred file blobs in the paired container.	C n
19007	AzureQueueBlobDownloadFailed WARN	"Azure Blob: failed to download" with <code>blob</code> . Failed to download a deferred file blob.	C n
19008	AzureQueueBlobHandlerError WARN	"Azure Blob: handler returned error" with <code>blob</code> . File handler rejected a downloaded blob.	C e
19009	AzureQueueBlobDeleteFailed WARN	"Azure Blob: failed to delete after processing" with <code>blob</code> . Failed to delete a blob after successful processing.	C

Code	Name / Level	Log message and context	T
19010	AzureQueueSPCredentialFailed ERROR	"Azure Queue: service principal credential failed" with <code>tenant_id</code> , <code>client_id</code> , and a sanitized <code>error</code> . <code>azidentity.NewClientSecretCredential</code> rejected the supplied Service Principal credentials at provider construction time.	V F e s n a c

nats_provider.go (20000-20999)

Emitted from the NATS transport provider and its JetStream Object Store file handling.

Code	Name / Level	Log message and context	Troubleshooting
20000	NATSDownloadFileFailed ERROR	"Failed to download file from object store" with <code>key</code> . JetStream Object Store download failed.	Check JetStream health on the NATS broker (<code>nats stream ls</code>), confirm the object store bucket exists, and check the <code>key</code> has not been pruned.
20001	NATSFileHandlerError WARN	"File handler returned error" with <code>key</code> . Downstream file handler rejected a downloaded object.	Correlate with the adjacent <code>15041 - 15045</code> inbound file error.
20002	NATSDisconnected WARN	NATS client disconnect callback with <code>name</code> . Log message is generated by the NATS client library.	Transient if followed by 20003 soon after. Persistent disconnects mean the broker is unreachable or the client is being forcibly kicked (auth rotation, account cap).

Code	Name / Level	Log message and context	Troubleshooting
20003	NATSReconnected INFO	NATS client reconnect callback with <code>name</code> . Informational.	Informational, confirms recovery after a 20002 event.

prompt.go (21000-21999)

Emitted from the inbound prompt subsystem. Prompts are interactive posts (Accept / Block buttons) shown when a new inbound connection tries to deliver to a team or channel that has not yet opted in.

Code	Name / Level	Log message and context	Troubleshooting
21000	PromptGetConnPromptFailed ERROR	"Failed to get connection prompt" with <code>team_id</code> , <code>conn</code> . Failed to read the stored team-level prompt state.	Inspect the KV store error.
21001	PromptGetTownSquareFailed ERROR	"Failed to get town-square for prompt" with <code>team_id</code> . Failed to find town-square when posting a team-level prompt.	Verify the team has a town-square channel. Rare; only occurs if the default channel was deleted, which Mattermost normally prevents.
21002	PromptCreatePostFailed ERROR	"Failed to create connection prompt post" with <code>team_id</code> , <code>conn</code> . Failed to post the prompt message.	Confirm the bot can post in town-square and the post payload does not violate server limits.

Code	Name / Level	Log message and context	Troubleshooting
21003	PromptSavePromptFailed ERROR	"Failed to save connection prompt" with <code>team_id</code> , <code>conn</code> . Failed to save the prompt state after posting.	Inspect the KV store error.
21004	PromptAcceptGetPromptFailed ERROR	"Failed to get connection prompt" with <code>team_id</code> , <code>conn</code> . Accept handler could not load the stored prompt state.	Inspect the KV store error. The admin may need to click Accept again after the store recovers.
21005	PromptDeletePromptFailed ERROR	"Failed to delete connection prompt" with <code>team_id</code> , <code>conn</code> . Failed to delete the prompt state after Accept.	Prompt may reappear on the next inbound message until the delete succeeds. Use <code>reset-prompt</code> to manually clear.
21006	PromptBlockGetPromptFailed ERROR	"Failed to get connection prompt" with <code>team_id</code> , <code>conn</code> . Block handler could not load state.	Inspect the KV store error.
21007	PromptSetBlockedFailed ERROR	"Failed to update connection prompt to blocked" with <code>team_id</code> , <code>conn</code> . Failed to mark the prompt as blocked.	Inspect the KV store error.
21008	PromptGetChanPromptFailed ERROR	"Failed to get channel connection prompt" with <code>channel_id</code> ,	Inspect the KV store error.

Code	Name / Level	Log message and context	Troubleshooting
		<code>conn</code> . Failed to read channel-level prompt state.	
21009	PromptCreateChanPostFailed ERROR	"Failed to create channel connection prompt post" with <code>channel_id</code> , <code>conn</code> . Failed to post the channel-level prompt.	Confirm the bot can post in the target channel.
21010	PromptSaveChanPromptFailed ERROR	"Failed to save channel connection prompt" with <code>channel_id</code> , <code>conn</code> . Failed to save channel-level prompt state.	Inspect the KV store error.
21011	PromptChanAcceptGetFailed ERROR	"Failed to get channel connection prompt" with <code>channel_id</code> , <code>conn</code> . Channel-level Accept handler failed to load state.	Inspect the KV store error.
21012	PromptDeleteChanPromptFailed ERROR	"Failed to delete channel connection prompt" with <code>channel_id</code> , <code>conn</code> . Failed to delete channel-level prompt state after Accept.	Use <code>reset-channel-prompt</code> to clear manually.
21013	PromptChanBlockGetFailed ERROR	"Failed to get channel connection prompt" with	Inspect the KV store error.

Code	Name / Level	Log message and context	Troubleshooting
		<code>channel_id</code> , <code>conn</code> . Channel-level Block handler failed to load state.	
21014	PromptSetChanBlockedFailed ERROR	"Failed to update channel connection prompt to blocked" with <code>channel_id</code> , <code>conn</code> . Failed to set channel-level blocked state.	Inspect the KV store error.
21015	PromptGetPostForUpdateFailed ERROR	"Failed to get prompt post for update" with <code>post_id</code> . Could not load the prompt post to update its displayed message after user action.	Post may have been deleted. Non-fatal; the state is still updated.
21016	PromptUpdatePostFailed ERROR	"Failed to update prompt post" with <code>post_id</code> . Could not update the prompt post to reflect the user's Accept/Block action.	Check bot post permissions. Non-fatal; the prompt state change is persisted even if the visible post still shows the old buttons.

retry_dispatch.go (22000-22999)

Emitted from the retry queue that holds inbound messages whose parent post has not yet been received. A background ticker reprocesses the queue.

Code	Name / Level	Log message and context	Troubleshooting
22000	RetryDispatchSucceeded WARN	"Missing message: retry succeeded" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>attempt</code> . A queued message was successfully dispatched after its parent arrived.	Informational for operators but logged at WARN so it is visible. Rising <code>attempt</code> counts indicate systemic out-of-order delivery.
22001	RetryDispatchStillMissing WARN	"Missing message: still missing after retry" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>attempt</code> , <code>next_retry_in</code> . Parent is still missing; the message stays queued.	Expected for a few attempts during out-of-order delivery. If this repeats past <code>attempt</code> = max-retries, see 22004.
22002	RetryDispatchDropMaxAge ERROR	"Missing message: dropped, exceeded max age" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>age</code> . Parent never arrived within max age; message dropped.	The parent post never made it through. Check the sender's logs for the <code>remote_post_id</code> and see whether an earlier error (15xxx, 16xxx, or sender-side) blocked it. Consider raising the max-age setting if senders are legitimately slow.
22003	RetryDispatchDropUnmarshal ERROR	"Missing message: dropped, payload unmarshal failed on retry" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> . A	Usually indicates corruption at rest or a codec version mismatch. The envelope was valid when first received, so

Code	Name / Level	Log message and context	Troubleshooting
		queued message failed to unmarshal on retry.	investigate memory or KV store corruption.
22004	RetryDispatchDropMaxRetries ERROR	"Missing message: dropped after max retries" with <code>conn</code> , <code>type</code> , <code>remote_post_id</code> , <code>attempts</code> . A queued message exceeded the max retry count.	Parent cannot be resolved. Investigate whether the parent post is being filtered out (e.g., blocked prompt, inactive channel, unresolved user) rather than delayed.

store/caching.go (23000-23999)

Emitted from the in-memory cache that sits in front of the KV store on each node.

Code	Name / Level	Log message and context	Troubleshooting
23000	StoreCachePublishInvalidationFailed WARN	"Failed to publish cache invalidation event" with <code>event</code> , <code>key</code> . Could not publish a cluster cache invalidation message after a KV write.	In a multi-node Mattermost cluster, other nodes may serve stale data until the next TTL (default 30 seconds). Check that the Mattermost cluster plugin message bus is healthy. Single-node deployments can ignore this code.

request.go (24000-24999)

Emitted from the team admin connection request workflow. When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can submit link requests that System Admins approve or deny via interactive DM buttons. These codes cover request creation, approval, denial, cancellation, and notification delivery.

Code	Name / Level	Log message and context	Troubleshooti
24000	RequestGetFailed ERROR	"Failed to get connection request" with <code>team_id</code> , <code>conn_key</code> . Also used during cancellation (WARN level): "Failed to get connection request for cancellation". KV store lookup for an existing pending request failed.	Check the <code>error</code> field f the underlying KV store error. Usually transient. If repeated, chec KV store health The request creation or cancellation w be aborted.
24002	RequestGetDMChannelFailed ERROR	"Failed to get DM channel for connection request" with <code>admin_id</code> . Could not open a DM channel between the bot and a System Admin to deliver the request notification.	Verify the bot user exists and the admin account is active. This admin will not receive the request DM, b others still will all admins fail, the request ma be saved with notification posts.
24003	RequestCreatedMPostFailed ERROR	"Failed to create DM post for connection request" with <code>admin_id</code> . The DM channel was opened but the post with	Check Mattermost server health and post

Code	Name / Level	Log message and context	Troubleshooti
		Approve/Deny buttons could not be created.	creation permissions. This admin will not see the request. If all admins fail, the request will be saved but no one can act on it from the UI.
24004	RequestSaveFailed ERROR	"Failed to save connection request" with <code>team_id</code> , <code>conn_key</code> . Could not persist the pending request to the KV store after DM posts were created.	The DM posts will be cleaned up automatically. Inspect the KV store error. The Team Admin will see a generic "failed to save message. Retry the command.
24005	RequestApproveGetFailed ERROR	"Failed to get connection request for approve" with <code>team_id</code> , <code>conn_key</code> . When a System Admin clicks Approve, the handler could not load the pending request from KV store.	Check KV store health. The admin will see "Failed to check request status in the post action response". The request remains pending and can be retried.

Code	Name / Level	Log message and context	Troubleshooti
24006	RequestApproveExecFailed ERROR	"Failed to execute approved connection request" with <code>team_id</code> , <code>conn_key</code> . The request was approved but the underlying <code>initTeamForCrossGuard</code> call failed.	Inspect the nested error message. Common causes: team deleted, connection config change KV store error. The request remains pending. Fix the underlying issue and have the admin click Approve again
24007	RequestApproveDeleteFailed ERROR	"Failed to delete approved connection request" with <code>team_id</code> , <code>conn_key</code> . The connection was linked successfully but the pending request record could not be cleaned up.	Non-critical: the connection is active. The stale request record will be detected as "no longer active" if someone click Approve/Deny again. No action needed unless it persists.
24008	RequestApproveNotifyFailed ERROR	"Failed to get DM channel for requester notification" or "Failed to notify requester" with <code>requester_id</code> . Could not send the approval/denial notification DM to the requesting Team Admin.	Check that the requester account still exists and the bot can open a DM. The connection action (approve/deny) was still

Code	Name / Level	Log message and context	Troubleshooti
			completed successfully; only the notification wa lost.
24009	RequestDenyDialogFailed ERROR	"Failed to open deny dialog" with <code>team_id</code> , <code>conn_key</code> . The Deny button was clicked but the interactive dialog (for entering a reason) could not be opened.	Check Mattermost server health. The request remains pending. The admin can try clicking Deny again or use th Approve butto instead.
24010	RequestDenyGetFailed ERROR	"Failed to get connection request for deny submit" with <code>team_id</code> , <code>conn_key</code> . After the deny dialog was submitted, the handler could not load the pending request.	Check KV stor health. The denial was not processed. Th admin can retr from the origin DM buttons.
24011	RequestDenyDeleteFailed ERROR	"Failed to delete denied connection request" with <code>team_id</code> , <code>conn_key</code> . Also used during cancellation (WARN level): "Failed to delete connection request during cancellation". The deny or cancel succeeded logically but the KV record was not removed.	Non-critical: same as 2400 The stale reco will be detecte as "no longer active" on nex interaction.

Code	Name / Level	Log message and context	Troubleshooti
24012	RequestDenyNotifyFailed ERROR	Reserved for denial notification failures. Currently shares the RequestApproveNotifyFailed code path (24008) for requester notification.	See 24008. Th code is allocat for future use deny notifications need a distinct code path.
24013	RequestUpdatePostFailed WARN	"Failed to get request post for update" or "Failed to update request post" with <code>post_id</code> . After approval, denial, or cancellation, the handler could not update one of the admin DM posts to remove the Approve/Deny buttons.	Non-critical: tl request action was still processed. Th admin may see stale buttons, but clicking them will show "This request i no longer active." Post may have beer deleted manually.
24014	RequestNoSystemAdmins WARN	"No system admins available to review connection request" with <code>team_id</code> , <code>conn_key</code> . The Team Admin submitted a request but the system has no admin users to notify.	Ensure at least one System Admin account exists and is active. The request was n created. The Team Admin sees "no syste admins availab to review your request."
24015	RequestConnRemovedFromConfig WARN	"Connection removed from config while request was pending" with <code>team_id</code> , <code>conn_key</code> . A System Admin	The request is automatically cancelled and the requester i

Code	Name / Level	Log message and context	Troubleshooti
		clicked Approve but the connection no longer exists in plugin configuration.	notified. The connection must be re-added to the configuration before a new request can be submitted.
24016	RequestConfirmDMFailed WARN	"Failed to get DM channel for requester confirmation" or "Failed to send requester confirmation DM" with <code>user_id</code> . After DM notifications were sent to System Admins, the confirmation DM to the requesting Team Admin could not be delivered.	Non-critical: the request was created and admins were notified. Only the requester's confirmation message was lost. Check that the requester account is active and the bot can open a DM.
24017	RequestNoAdminsNotified ERROR	"Failed to notify any system admin about connection request" with <code>team_id</code> , <code>conn_key</code> . DM delivery failed for every System Admin. The reserved request is cleaned up automatically.	Ensure at least one System Admin account is active and the bot can open DM channels. The Team Admin sees "failed to notify any system admin about your request." Retry after fixing admin account

channel_request.go (25000-25999)

Emitted from the channel admin connection request workflow. When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can submit channel-level link requests that Team Admins approve or deny via interactive DM buttons. These codes cover request creation, approval, denial, cancellation, and notification delivery.

Code	Name / Level	Log message and context	Troubleshooting
25000	<code>ChanRequestGetFailed</code> ERROR	"Failed to get channel connection request" with <code>channel_id</code> , <code>conn_key</code> . Also used during cancellation (WARN level): "Failed to get channel connection request for cancellation". KV store lookup for an existing pending channel request failed.	Check the KV store for the underlying error. If transient. If check KV store. The request cancellation aborted.
25002	<code>ChanRequestGetDMChannelFailed</code> ERROR	"Failed to get DM channel for channel connection request" with <code>admin_id</code> . Could not open a DM channel between the bot and a Team Admin to deliver the request notification.	Verify the bot exists and the Admin account is active. This user may not receive DM, but other users can.
25003	<code>ChanRequestCreatedMPPostFailed</code> ERROR	"Failed to create DM post for channel connection request" with <code>admin_id</code> . The DM channel was opened but the post with Approve/Deny buttons could not be created.	Check Mattermost server health. DM creation per user. This admin may not have the request.
25004	<code>ChanRequestSaveFailed</code> ERROR	"Failed to save channel connection request" with <code>channel_id</code> , <code>conn_key</code> . Could not persist the pending channel request to the KV store. Also used (WARN level)	Inspect the error. The Channel Admin will see a generic "fail message. Recommend a command.

Code	Name / Level	Log message and context	Troubleshooting
		when updating the stored request with post IDs fails.	
25005	ChanRequestApproveGetFailed ERROR	"Failed to get channel connection request for approve" with <code>channel_id</code> , <code>conn_key</code> . Also used when verifying team connections during approval: "Failed to get team connections during channel request approve" with <code>team_id</code> .	Check KV store. The admin view shows error in the log response. The request remains pending and can be retrieved.
25006	ChanRequestApproveExecFailed ERROR	"Failed to execute approved channel connection request" with <code>channel_id</code> , <code>conn_key</code> . The request was approved but the underlying <code>initChannelForCrossGuard</code> call failed.	Inspect the error message. Common cause is channel deletion. Check KV store error and underlying issue. Have the admin approve again.
25007	ChanRequestApproveDeleteFailed ERROR	"Failed to delete channel connection request during approve" with <code>channel_id</code> , <code>conn_key</code> . The pending request record could not be removed after approval started (delete-first pattern).	The request should be aborted. If it sees "Failed request. Please try again." Retry after checking KV health.
25008	ChanRequestApproveNotifyFailed ERROR	Reserved for approval notification failures. Currently uses the shared <code>notifyRequester</code> and <code>updateRequestPosts</code> helpers.	See the <code>updateReq</code> and <code>notifyReq</code> error handling. It is critical if the request was linked successfully.

Code	Name / Level	Log message and context	Troubleshooting
25009	ChanRequestDenyDialogFailed ERROR	"Failed to open channel deny dialog" with <code>channel_id</code> , <code>conn_key</code> . The Deny button was clicked but the interactive dialog could not be opened.	Check Matter server health. The deny request remains pending. The user can try clicking again.
25010	ChanRequestDenyGetFailed ERROR	"Failed to get channel connection request for deny" or "Failed to get channel connection request for deny submit" with <code>channel_id</code> , <code>conn_key</code> . The handler could not load the pending channel request during denial.	Check KV state. The denial was not processed. The user can retry from the original DM.
25011	ChanRequestDenyDeleteFailed ERROR	"Failed to delete denied channel connection request" with <code>channel_id</code> , <code>conn_key</code> . Also used during cancellation (WARN level): "Failed to delete channel connection request during cancellation".	Non-critical. The record will be deleted as "no longer active" on next interaction.
25012	ChanRequestDenyNotifyFailed ERROR	Reserved for denial notification failures. Currently shares the <code>notifyRequester</code> and <code>updateRequestPosts</code> code paths.	See 25008. This is allocated for use if deny notifications and updates are distinct code paths.
25013	ChanRequestUpdatePostFailed WARN	Reserved for post update failures during channel request lifecycle. Currently uses the shared <code>updateRequestPosts</code> helper which logs via <code>RequestUpdatePostFailed</code> (24013).	Non-critical. The request action was processed. The user may see status but clicking to show "This is no longer active".

Code	Name / Level	Log message and context	Troubleshooting
25014	ChanRequestNoTeamAdmins WARN	"No team admins available to review channel connection request" with <code>channel_id</code> , <code>team_id</code> , <code>conn_key</code> . The Channel Admin submitted a request but no Team Admins exist for this team.	Ensure at least one Team Admin is active in the team. The request was not created until a Team Admin was not created. Channel Admins can submit a request but no team admins are available to review the request."
25015	ChanRequestConnRemovedFromConfig WARN	"Connection removed from config while channel request was pending" with <code>channel_id</code> , <code>conn_key</code> . A Team Admin clicked Approve but the connection no longer exists in plugin configuration.	The request was automatically removed and the requester was notified. The connection was removed and the request was added to the configuration. A new request was submitted.
25016	ChanRequestConfirmDMFailed WARN	"Failed to get DM channel for channel requester confirmation" or "Failed to send channel requester confirmation DM" with <code>user_id</code> . After DM notifications were sent to Team Admins, the confirmation DM to the requesting Channel Admin could not be delivered.	Non-critical request was not confirmed and Team Admins were notified. On the requester's confirmation was lost. Channel Admins can confirm the request is active.
25017	ChanRequestNoAdminsNotified ERROR	"Failed to notify any team admin about channel connection request" with <code>channel_id</code> , <code>team_id</code> , <code>conn_key</code> . DM delivery failed for every Team Admin. The reserved request is cleaned up automatically.	Ensure at least one Team Admin is active and the requester has an open DM channel. Channel Admins can submit a request but no team admin is available to review the request." Re

Code	Name / Level	Log message and context	Troubleshooting
			fixing Team accounts.
25018	ChanRequestGetTeamAdminsFailed ERROR	"Failed to get team members for admin lookup" with <code>team_id</code> . The paginated team member lookup to find Team Admins failed. Also logged (WARN level) when an individual user lookup within the pagination loop fails: "Failed to get user during team admin lookup" with <code>user_id</code> .	Check Matt's health. The Admin will send a generic "fail" up team address. Retry the connection. Individual users fail, those users are skipped but still be notified.

azure_servicebus_provider.go (26000-26999)

Emitted from the Azure Service Bus provider, which uses an Azure Service Bus queue (PeekLock receive mode) for envelopes and an optional Azure Blob Storage sidecar container for file attachments. Service Bus send failures are returned to the caller rather than logged by the provider; upstream retry dispatch logs those via its own error code. The code `APIAzureServiceBusTestFailed` (26006) is emitted from `api.go` but is grouped here because it reports a Service Bus connection failure.

Code	Name / Level	Log message and context	Troubleshooting
26000	ServiceBusSendFailed WARN	"Service Bus: could not create blob container (may already exist)" with <code>container</code> . Idempotent blob sidecar container create returned an error at provider startup. (Reserved for send-path logging; <code>Publish</code> currently returns sanitized errors to the caller rather than logging them locally.)	Use the container (create) exception vector. Create a container. Currently returns sanitized errors to the caller rather than logging them locally.

Code	Name / Level	Log message and context	Tr
26001	ServiceBusReceiveFailed ERROR/WARN	"Service Bus receive failed" (ERROR) with <code>queue</code> , and (WARN) "Service Bus blob list failed", "Service Bus blob: failed to download", "Service Bus blob: handler returned error", "Service Bus blob: failed to delete after processing", each with the corresponding <code>container</code> or <code>blob</code> field. Covers both the Service Bus receive loop and the blob sidecar poll.	Ch re Se na en re or int blo re blo
26002	ServiceBusCompleteFailed WARN	"Service Bus complete failed; message will redeliver" or "Service Bus complete failed on empty-body drop; message will redeliver" with <code>queue</code> and <code>message_id</code> . Complete (settle) call against a received message failed.	Th ex mi re au int idi idi so su Re pc ne pe
26003	ServiceBusAbandonFailed WARN	"Service Bus abandon failed" with <code>queue</code> . Abandon call after a handler error failed; the lock will expire instead of being released explicitly.	Nc mi re loc Inv wi ev
26004	ServiceBusRedelivery WARN	"Service Bus message redelivered" with <code>queue</code> , <code>delivery_count</code> , <code>message_id</code> . A received message has a <code>DeliveryCount</code> > 1 , meaning Service Bus has redelivered it at least once.	Ex tra fai ra re

Code	Name / Level	Log message and context	Tr
			pc Se m to qu qu M is Az se me th
26005	ServiceBusMalformedBody WARN	"Service Bus message has empty body; completing to drop" with <code>queue</code> , <code>message_id</code> . A received message had a zero-length body. The plugin completes (drops) it to avoid an infinite redelivery loop.	Ca m so ur wr qu bu en
26006	APIAzureServiceBusTestFailed ERROR	"Azure Service Bus connection test failed" with <code>error</code> (sanitized). Emitted from <code>handleTestAzureServiceBusConnection</code> when the admin "Test Connection" button cannot peek the target queue.	Th alr SA/ Ty qu (p Az AF ur th sti pe ur en fir
26007	ServiceBusSPCredentialFailed ERROR	"Service Bus: service principal credential failed" with <code>tenant_id</code> , <code>client_id</code> , and a sanitized <code>error</code> .	Ve (G c

Code	Name / Level	Log message and context	Tr
		azidentity.NewClientSecretCredential rejected the supplied Service Principal credentials at provider construction time.	th c nc er sc cr

[Back to Overview](#)